DATA STRUCTURE AND ALGORITHM

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	G. HEMA HARSHINI
ROLL NO	AV.SC.U4CSE25108
YEAR/SEM/SECTION	2ST YEAR/SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	
NAME	DR.RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(S1. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	PROGRAM	PAGE NO	REMARKS
1.	**WEEK 1:** 1a) A rocket launch displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1, followed by "Launch!". 1b) Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively, write a recursive program to calculate p^n (power) where: p=principal growth factor, n=no. of years. 1c) An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function search for an employee ID in a list. 1d) In a logistics company, the no of possible ways to arrange n different parcels are calculated using the factorial formula(n!). Develop a recursive program to calculate the factorial of a given number. 1e) Write a recursive python program to print the 1st 'n' terms of Fibonacci series.	5-12	
2.	**WEEK 2:** 2a) Write a python program to implement linear search. 2b) (i) Write a python program to implement binary search with sorted array as an input. (ii) Write a python program to implement binary search without sorted array as an input.	12-17	
3.	**WEEK 3:** 3a) Write a python program to implement bubble sort by taking the input. 3b) Write a python program to implement insertion sort by taking the input. 3c) Write a python program to implement selection sort by taking the input.	17-23	
4.	**WEEK 4:** 4a) Write a python program to implement merge sort. 4b) Write a python program to implement quick sort.	23-28	
5.	**WEEK 5:** 5a) Write a python program to implement Singly Linked list with the below operation: i.Create Linked List ii.Insert beginning iii.Insert end		

	iv.Insert at index v.Delete by value vi.Delete first node vii.Delete last node viii.Count no.of nodes ix.Exit 5b) Write a python program to implement Doubly Linked list with the below operation: i.Create Linked List ii.Insert beginning iii.Insert end iv.Insert at index v.Delete by value vi.Delete first node vii.Delete last node viii.Count no.of nodes ix.Exit 5c) Write a python program to implement Circular Linked list with the below operation: i.Create Linked List ii.Insert beginning iii.Insert end iv.Insert at index v.Delete by value vi.Delete first node vii.Delete last node viii.Count no.of nodes ix.Exit	28-54	
6.	**WEEK 6:** 6a) Write a python program to implement stack operations such as push(),pop(),peek().Display() using an array. 6b) Write a python program to implement stack operations such as push(),pop(),peek().Display() using an Linked list.	54-61	
7.	**WEEK 7:** 7a) Write a python program to implement a queue and its operations such as enqueue,dequeue,peek and display with a menu driven approach.Implement using array. 7b) Write a python program to implement a queue and its operations such as enqueue,dequeue,peek and display with a menu driven approach.Implement using Singly Linked list. 7c) Write a python program to implement a queue and its operations such as enqueue,dequeue,peek and display with a menu driven approach.Implement using Circulated queue.	62-75	
8.	**WEEK 8:** 8a) Write a python program to implement a binary tree with its operations using array.	75-83	

	8b) Write a python program to implement a binary tree with its operations using linked list.		
9.	**WEEK 9:** 9a) Write a python program to implement a binary search tree with its operations and traversals.	83-94	
10.	**WEEK 10:** 10a) Write a python program to implement an AVL tree will all of its operation.	64-100	
11.	**WEEK 11:** 11a) Write a python program to implement hashing (separate chaining, linear probing and quadratic probing).	100-107	

WEEK-1

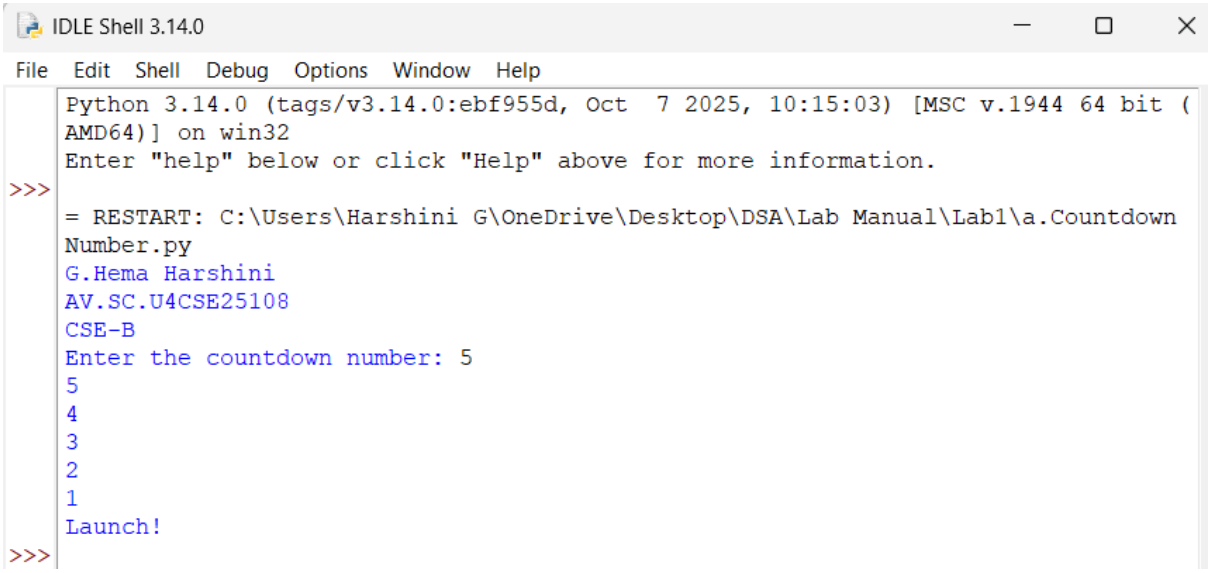
1a) **AIM:** A rocket launch displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1, followed by "Launch!".

CODE:

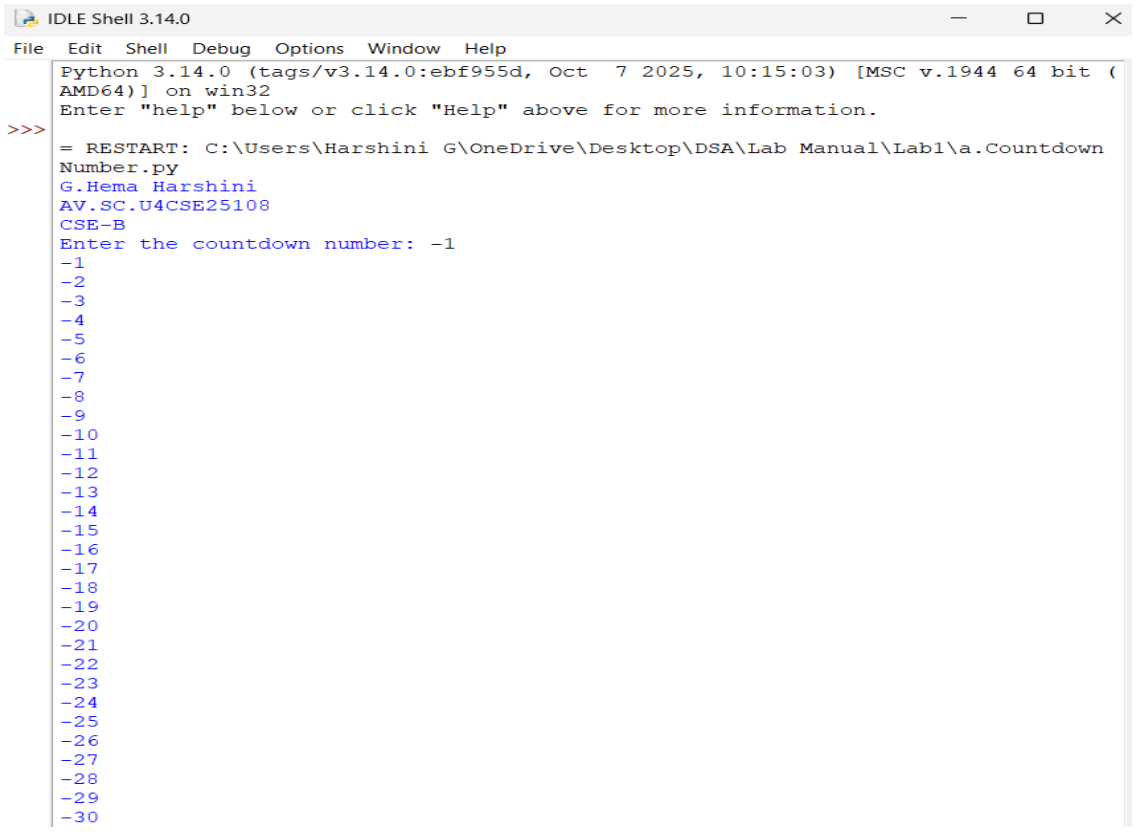
```
print("G.Hema Harshini")
print("AV.SC.U4CSE25108")
print("CSE-B")
def countdown(n):
    if n == 1:
        print(1)
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)

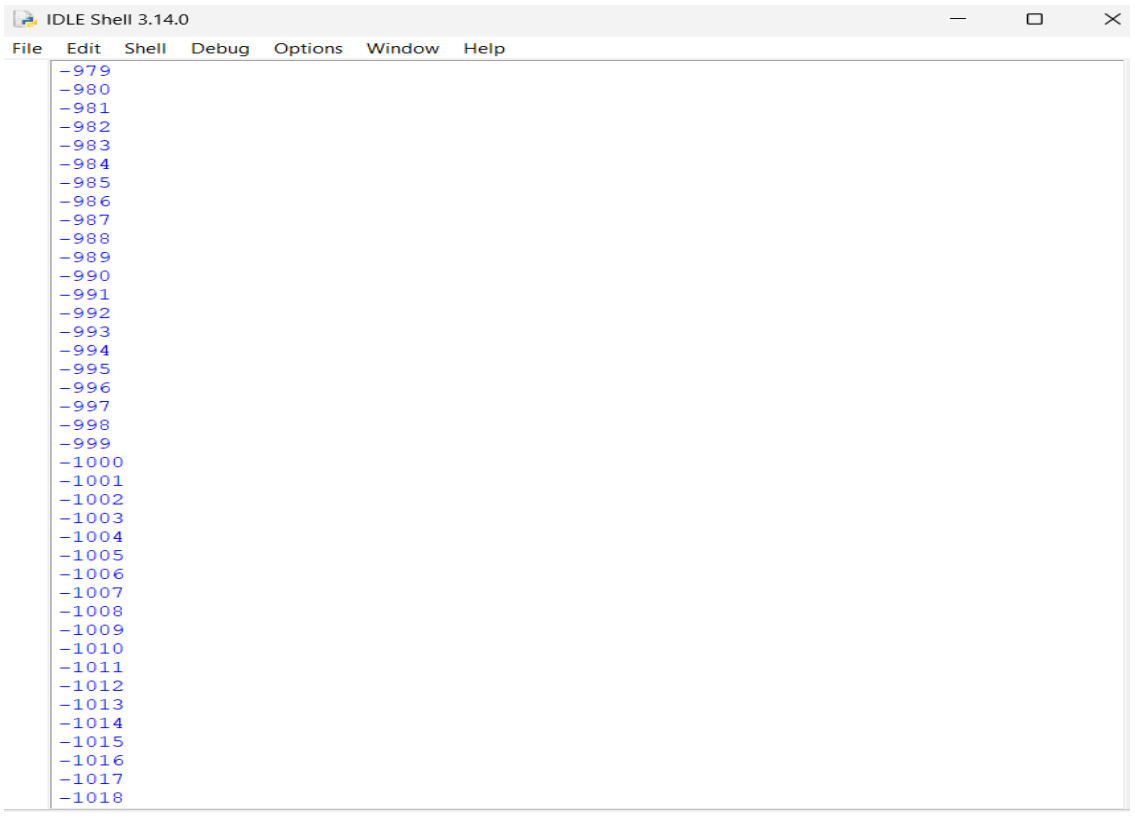
n = int(input("Enter the countdown number: "))
countdown(n)
```

OUTPUT(+VE):



OUTPUT(-VE):





Error Table:

Edge Case	Input	Error	Explanation
Large value of n	n== 1000	Recursive Error	May cause excessive recursive calls and a Recursion Error

Explanation:

1. Define the function: A function named countdown() is created. It takes n as the input.
2. If is the base case of the recursion. When n becomes 1, then function displays the statement in the print() function.
3. If n is not 1: function displays the statement in the print() function. The function calls itself with n - 1.
4. countdown(n): This calls the function using the number entered by the user.

1b) **AIM:** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively, write a recursive program to calculate p^n (power) where:

p=principal growth factor, n=no. of years.

CODE:

```
print("G.Hema Harshini")

print("AV.SC.U4CSE25108")

print("CSE-B")


def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

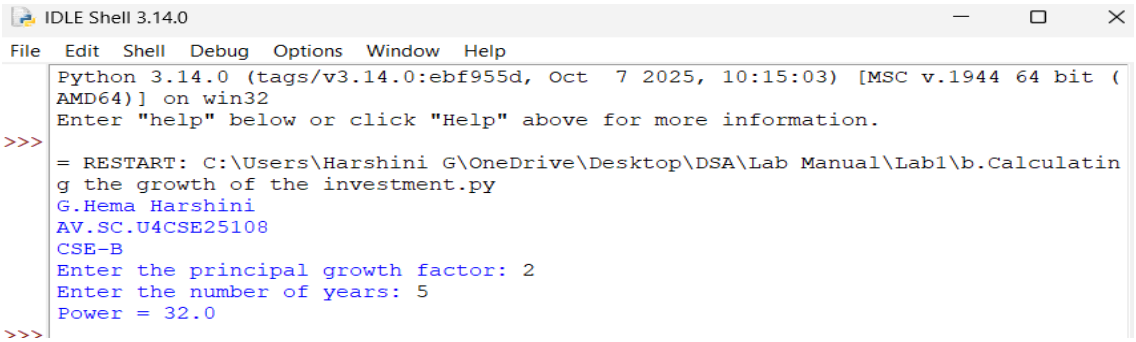

p = float(input("Enter the principal growth factor: "))

n = int(input("Enter the number of years: "))

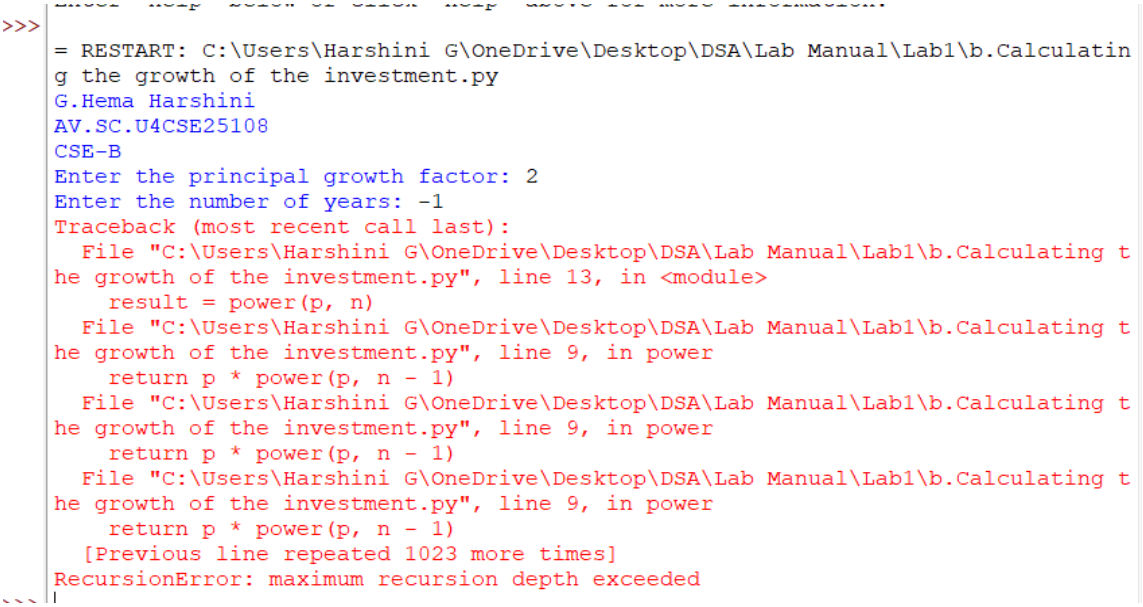
result = power(p, n)

print("Power =", result)
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Negative Number n	P==2,n== -1	Recursive Error	n keeps decreasing: -1, -2, -3..., so it never reaches the base case n=0.

Explanation:

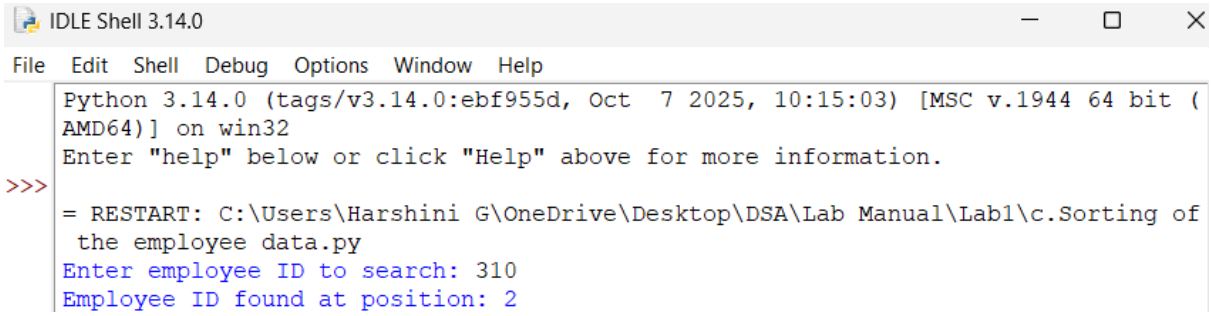
1. Define the function: A function named power() is created. It takes p and n as the input.
2. If is the base case of the recursion. When n becomes 1, then function displays the statement in the print() function.
3. If n is not 0, the function calls itself with n - 1.
4. The function is called using the values entered by the user.

1c) **AIM:** An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function search for an employee ID in a list.

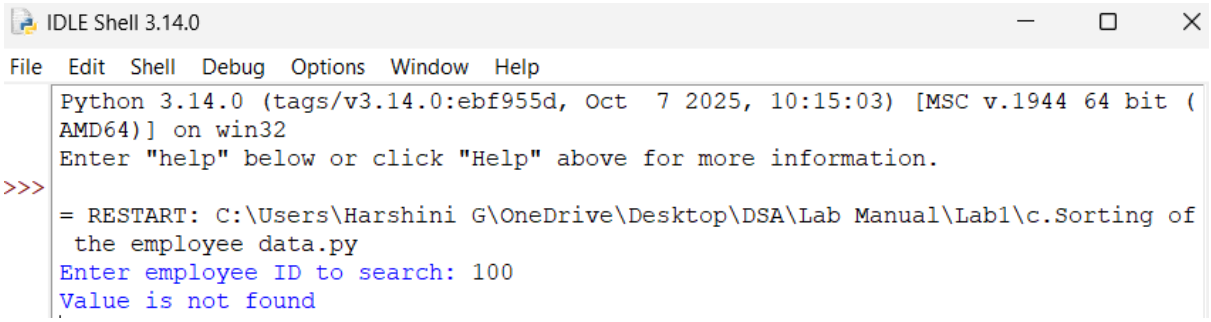
CODE:

```
def search(ids, key, index):
    if index == len(ids):
        return -1
    if ids[index] == key:
        return index
    return search(ids, key, index + 1)
ids = [101, 205, 310, 415, 520]
key = int(input("Enter employee ID to search: "))
result = search(ids, key, 0)
if result != -1:
    print("Employee ID found at position:", result)
else:
    print("Value is not found")
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Not an integer	a	Value is not found	a is an invalid key type

Explanation:

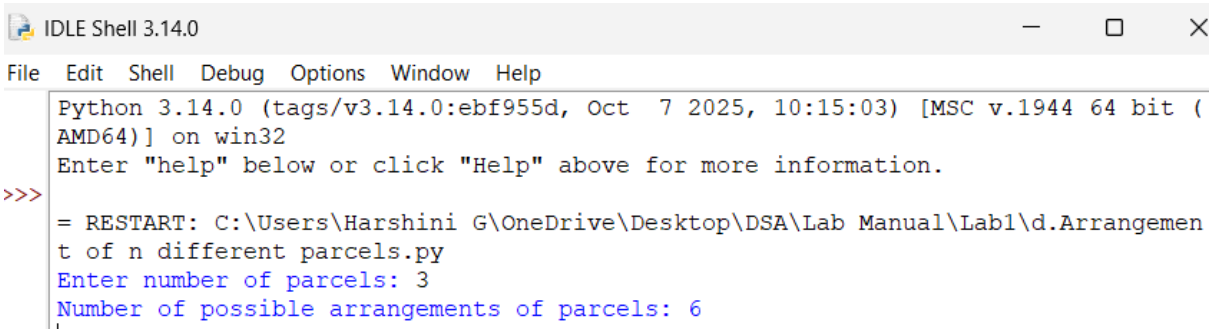
1. Define the function: A function named search() is created. It takes key as the input.
2. If is the base case of the recursion. When result not becomes -1, then function displays the statement in the print() function.
3. If result is not -1, the function displays the statement in the print() function.

1d) **AIM:** In a logistics company, the no of possible ways to arrange n different parcels are calculated using the factorial formula(n!). Develop a recursive program to calculate the factorial of a given number.

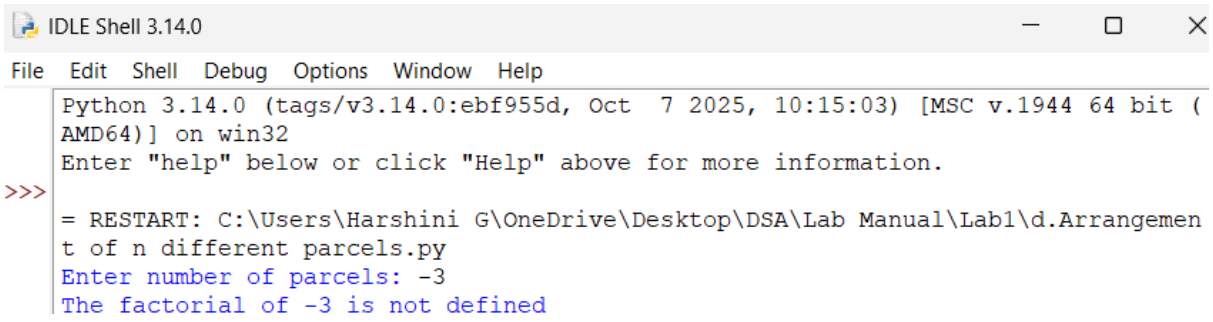
CODE:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
n = int(input("Enter number of parcels: "))
if n < 0:
    print(f"The factorial of {n} is not defined")
else:
    print("Number of possible arrangements of parcels:", factorial(n))
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Large number	1000	Recursion error	May cause excessive recursive calls and result in a Recursion Error.

Explanation:

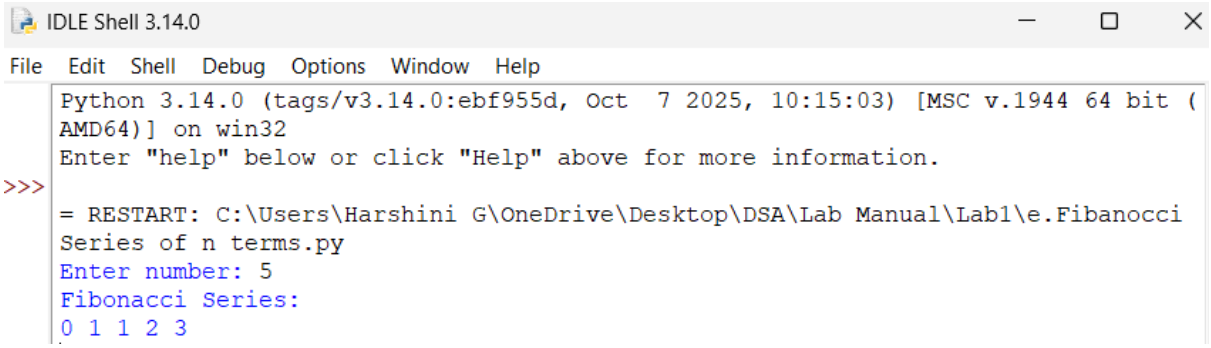
1. Define the function: A function named factorial() is created. It takes n as the input.
2. If is the base case of the recursion. When n becomes 0 or 1, then function displays the statement in the print() function.
3. If n is not 0 or 1, the function displays the statement in the print() function.

1e) **AIM:** Write a recursive python program to print the 1st 'n' terms of Fibonacci series.

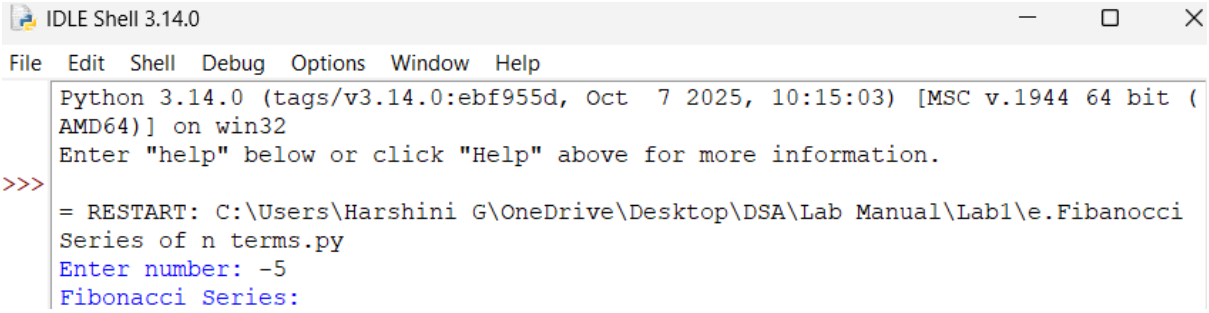
CODE:

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
n = int(input("Enter number: "))
print("Fibonacci Series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Empty input	-	No value entered	Display an input error and ask for a valid number.

Explanation:

1. Define the function: A function named fibanocci() is created. It takes n as the input.
2. If is the base case of the recursion. When n becomes less than 1, then it returns n.
3. If n is not less than 1, then it returns the function of the factorial of n-1 and n -2.
4. If for i in the range then display the fibanocci series.

WEEK-2

2a) **AIM:** Write a python program to implement linear search.

CODE:

```
arr = [10, 20, 30, 40, 50]

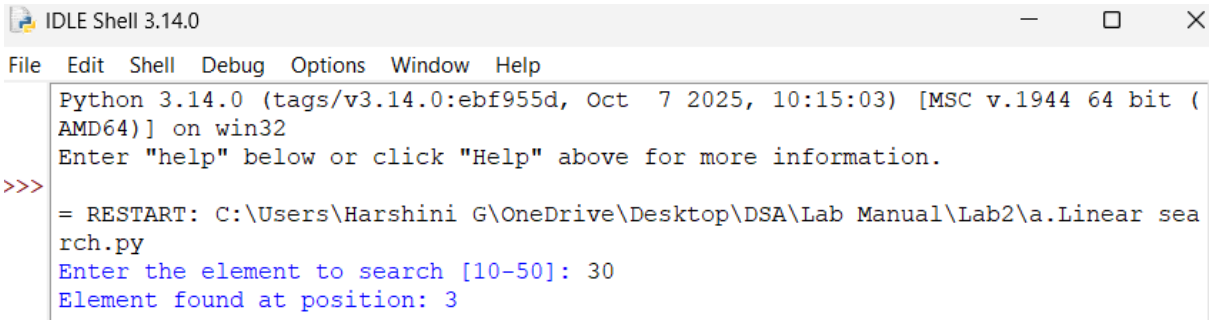
key = int(input("Enter the element to search [10-50]: "))

position = -1

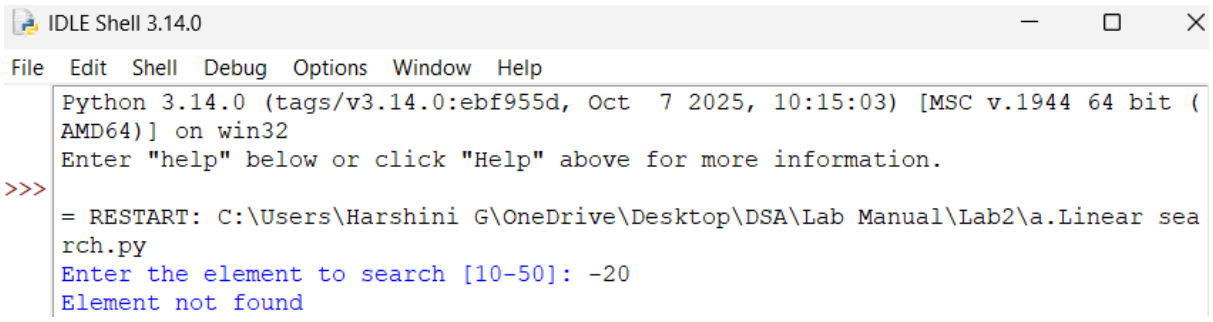
for i in range(len(arr)):
    if arr[i] == key:
        position = i
        break

if position != -1:
    print("Element found at position:", position + 1)
else:
    print("Element not found")
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Invalid position	Position is outside the array range	Element not found	Stop recursion and display "Element not found".

Explanation:

1. Take the input of the array. It takes key as the input.
2. If for i in the range then display the functions in it.
3. If is the base case of the recursion. When position is not 1, then function displays the statement in the print() function.
4. If n is not 1, then function displays the statement in the print() function.

2b)

i) **AIM:** Write a python program to implement binary search with sorted array as an input.

CODE:

```
n = int(input("Enter the number of elements: "))

arr = []

print("Enter elements in sorted order:")

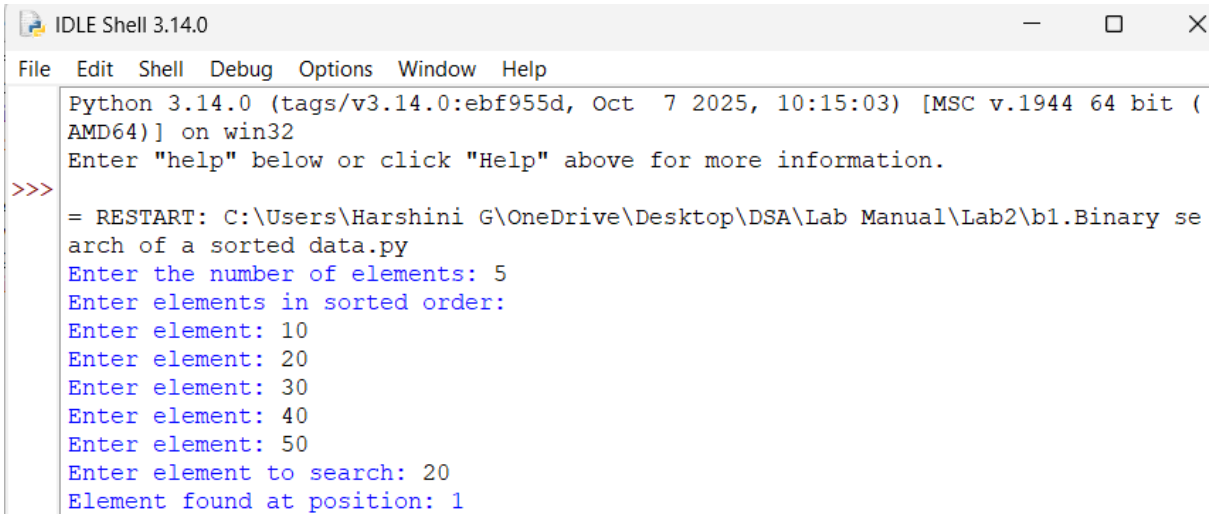
for i in range(n):

    arr.append(int(input("Enter element: ")))

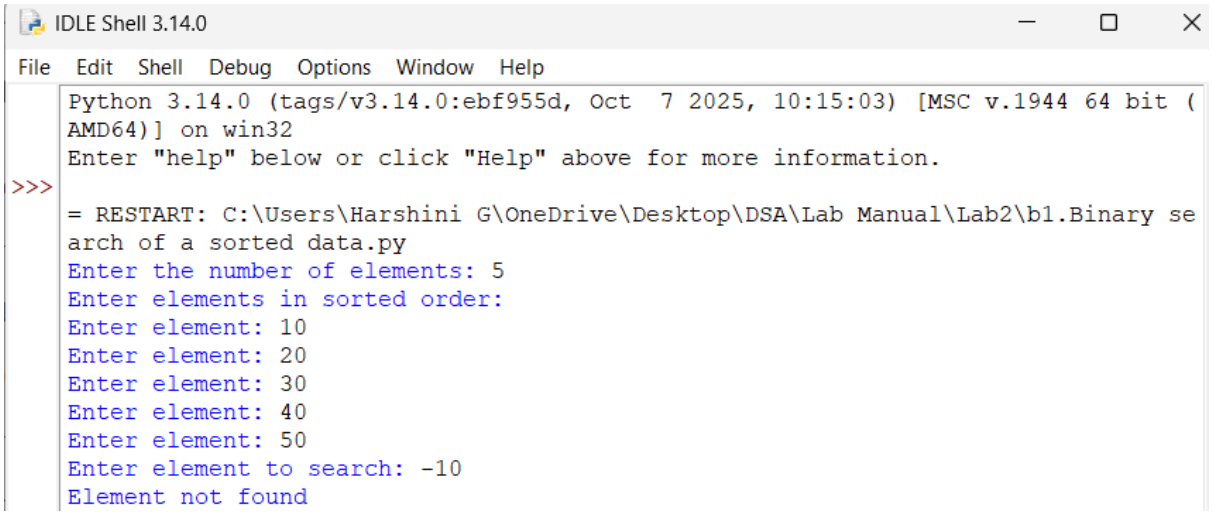
key = int(input("Enter element to search: "))
```

```
low = 0
high = n - 1
while low <= high:
    mid = (low + high) // 2
    if arr[mid] == key:
        print("Element found at position:", mid)
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Element not found")
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Invalid input	abc	Input Error	Display an input error because the key must be an integer.

Explanation:

1. The program takes the number of elements and then takes the elements of the array as input. The elements must be entered in sorted order.

2. The program takes the value key that has to be searched in the sorted array.

3. The variables low and high are initialized. low = 0 represents the first position, and high = n - 1 represents the last position of the array.

4. The program calculates the middle position using: mid = (low + high) // 2

5. If arr[mid] == key, the element is found and its position is displayed using the print() function.

6. If arr[mid] < key, the key may be present in the right half of the array. Therefore, low is changed to mid + 1.

7. If arr[mid] > key, the key may be present in the left half of the array. Therefore, high is changed to mid - 1.

8. The while low <= high condition repeatedly searches the middle portion until the element is found or there are no more elements to search.

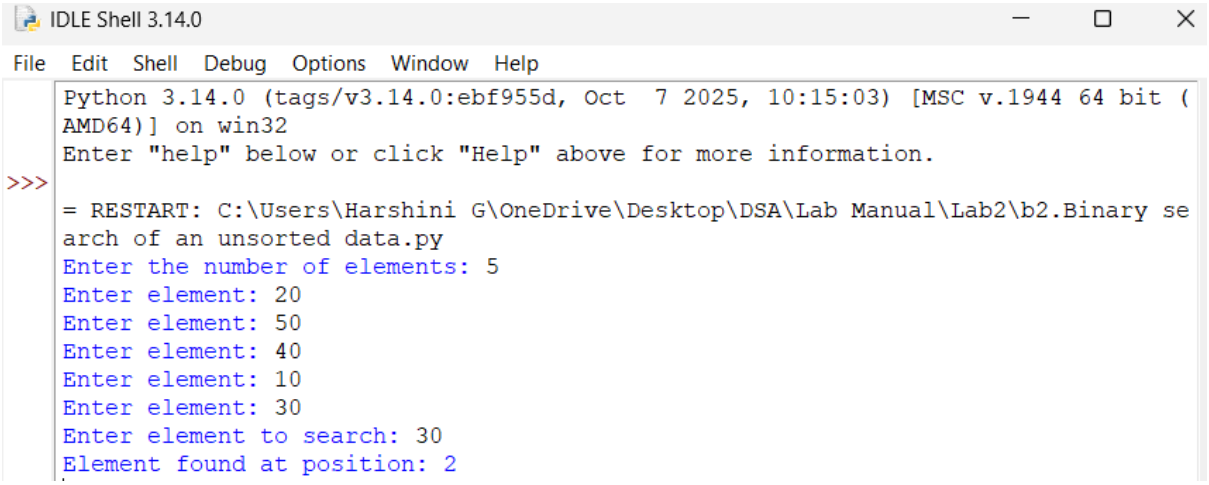
9. If the key is found, the program displays "Element found at position". If the search finishes without finding the key, the program displays "Element not found".

ii) **AIM:** Write a python program to implement binary search without sorted array as an input.

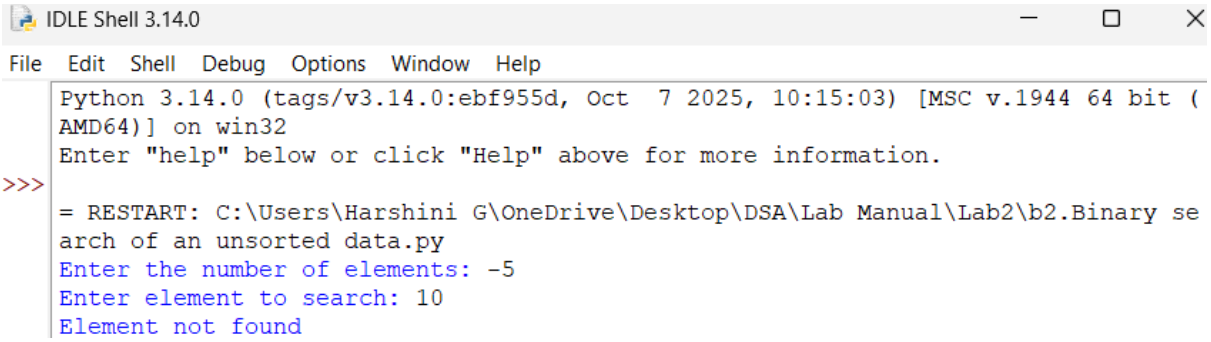
CODE:

```
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input("Enter element: ")))
arr.sort()
key = int(input("Enter element to search: "))
low = 0
high = n - 1
while low <= high:
    mid = (low + high) // 2
    if arr[mid] == key:
        print("Element found at position:", mid)
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Element not found")
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Negative number	-5	Element is not found	-5 is a negative number

Explanation:

1. The program first takes the number of elements and then takes the elements of the array from the user. The elements do not need to be entered in sorted order.

2. The sort() function is used to arrange the elements in ascending order.

arr.sort()

This is necessary because binary search works only on a sorted array.

3. After sorting, the program displays the sorted array using the print() function.

4. The program takes the value key that needs to be searched in the array.

5. The variables low and high are initialized. low = 0 represents the first position, and high = n - 1 represents the last position.

6. The middle position is calculated using:

mid = (low + high) // 2

7. If arr[mid] == key, the element is found and its position is displayed using the print() function.

8. If arr[mid] < key, the key may be present in the right half. Therefore:

low = mid + 1

9. If arr[mid] > key, the key may be present in the left half. Therefore:

high = mid - 1

10. The while low <= high loop continues dividing the search area into half until the key is found or there are no more elements to search.

11. If the key is found, the program displays "Element found at position". If the key is not found, it displays "Element not found".

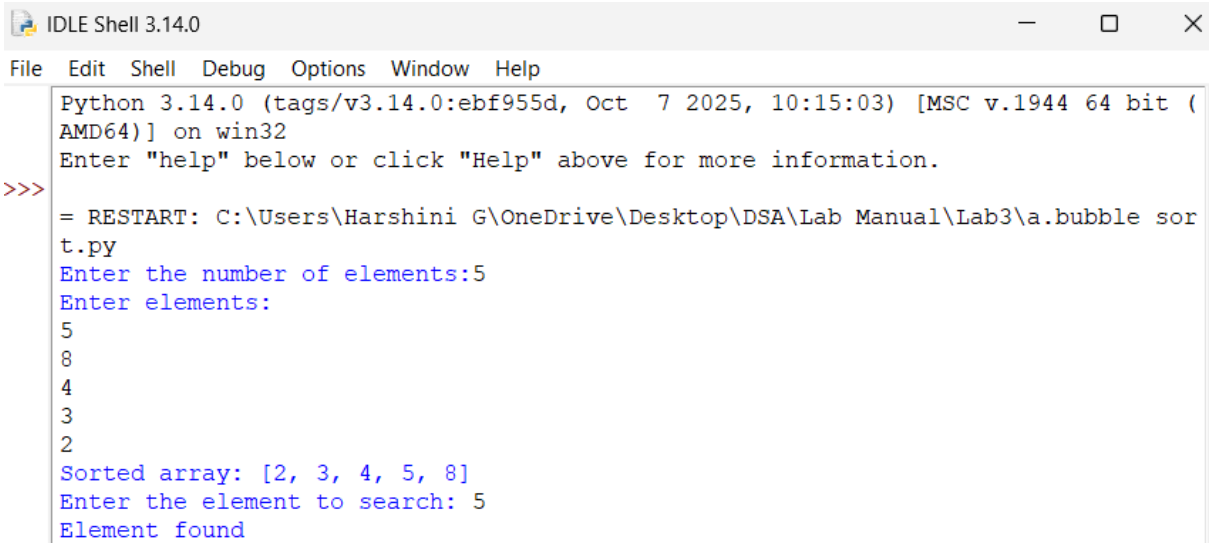
WEEK-3

3a) **AIM:** Write a python program to implement bubble sort by taking the input.

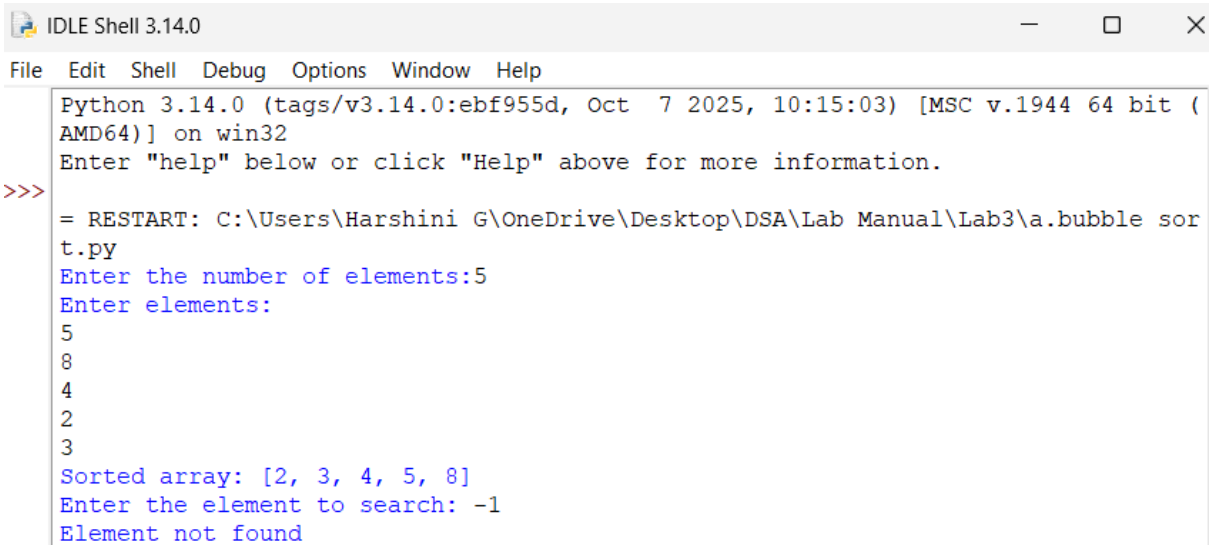
CODE:

```
def bubble_sort(arr):
    n=len(arr)
    for i in range(n-1):
        for j in range(n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]
    return arr
n=int(input("Enter the number of elements:"))
arr=[]
print("Enter elements: ")
for i in range(n):
    arr.append(int(input()))
print("Sorted array:", bubble_sort(arr))
key = int(input("Enter the element to search: "))
if key in arr:
    print("Element found")
else:
    print("Element not found")
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Invalid Syntax	,	Syntax error	‘,’ is not entered and there is syntax error.

Explanation:

1. The program first takes the number of elements n and then takes each element from the user using a for loop. The elements are stored in the list arr.

2. The outer for loop controls the number of passes required to sort the array.

for i in range(n - 1):

3. The inner loop compares two neighboring elements of the array.

for j in range(n - 1 - i):

4. If the current element is greater than the next element:

if arr[j] > arr[j + 1]:

the two elements need to be exchanged.

5. The elements are swapped using:

arr[j], arr[j + 1] = arr[j + 1], arr[j]

This moves the larger element toward the end of the array.

6. The inner loop continues comparing adjacent elements. After every pass, the largest unsorted element moves to its correct position at the end.

7. The outer loop continues until all elements are arranged in ascending order.

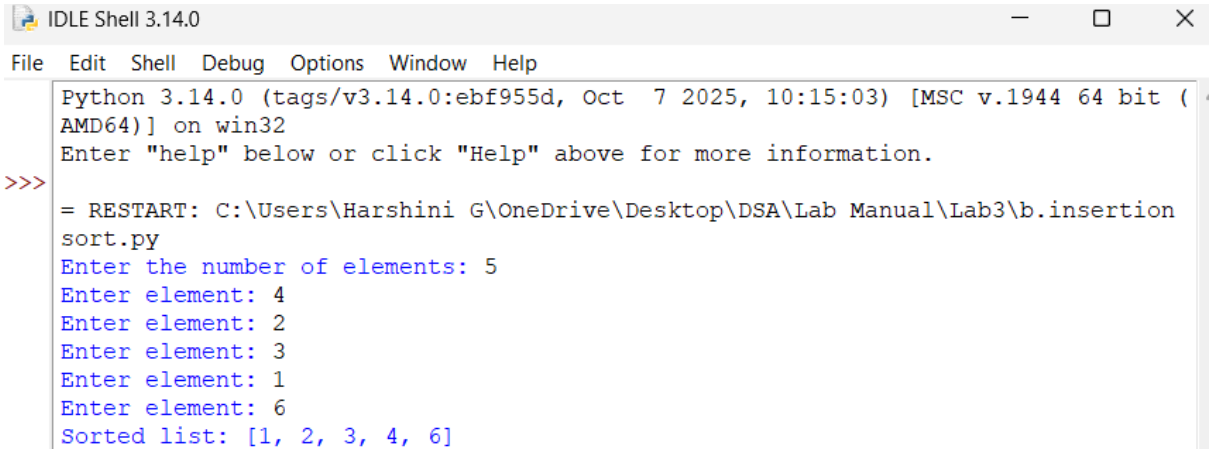
8. After sorting is completed, the program displays the sorted list

3b) **AIM:** Write a python program to implement insertion sort by taking the input.

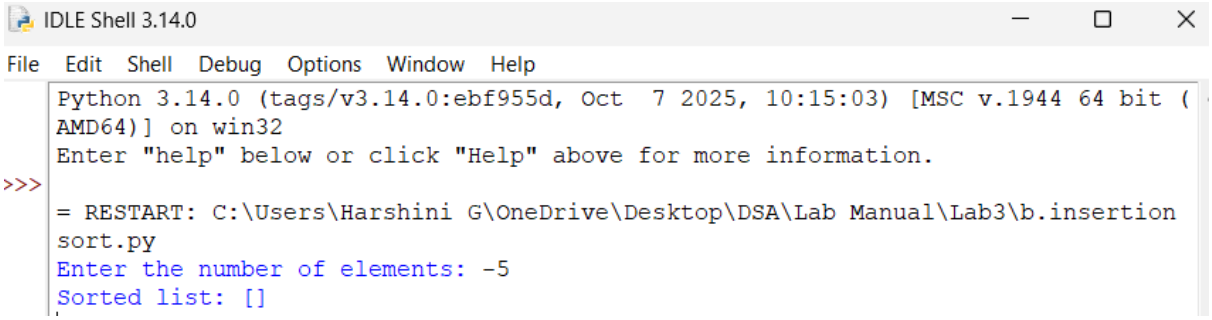
CODE:

```
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input("Enter element: ")))
for i in range(1, n):
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j = j - 1
    arr[j + 1] = key
print("Sorted list:", arr)
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Invalid Syntax	indentation	Indentation Error	there is an indentation error.

Explanation:

1. The program first takes the number of elements n and then takes each element from the user. The elements are stored in the list arr.

2. The for loop starts from index 1 because the first element is considered already sorted.

3. The current element is stored in the variable key.

key = arr[i]

This is the element that needs to be inserted into its correct position.

4. The variable j is set to the position just before the key.

j = i - 1

5. The while loop checks whether the previous element is greater than the key.

while j >= 0 and arr[j] > key:

6. If arr[j] is greater than key, it is moved one position to the right.

arr[j + 1] = arr[j]

Then j is decreased by 1 to check the next element.

7. When the correct position is found, the key is inserted using:

arr[j + 1] = key

8. The for loop continues with the next element. At each step, the sorted portion of the array becomes larger.

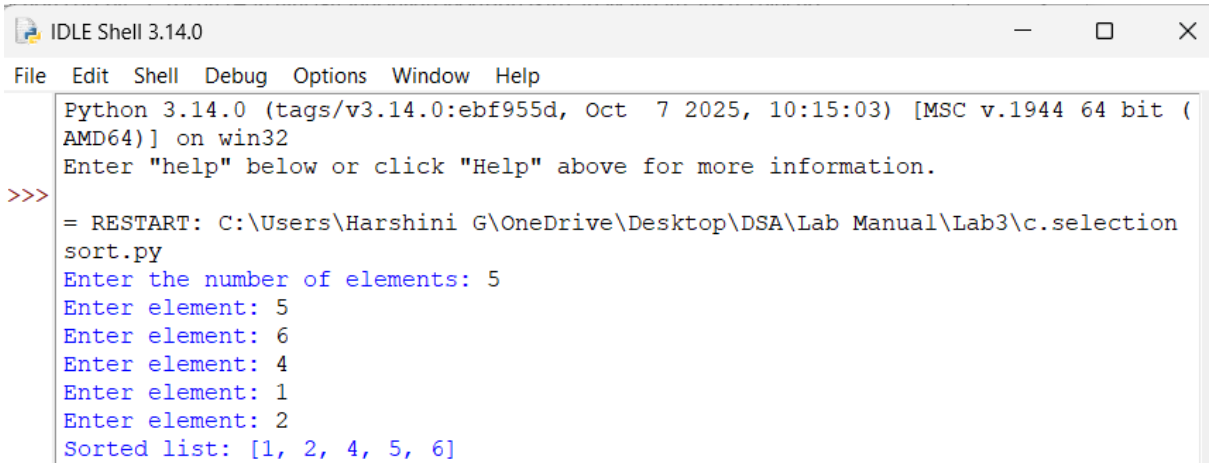
9. After all elements have been inserted into their correct positions, the program displays

3c) **AIM:** Write a python program to implement selection sort by taking the input.

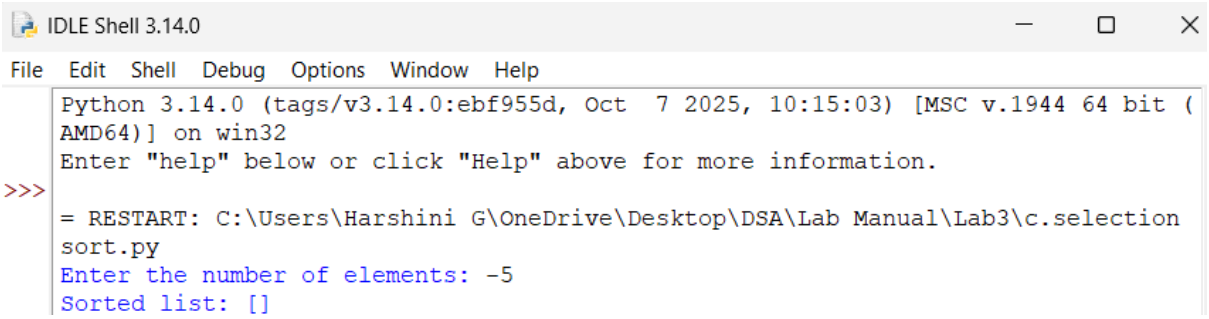
CODE:

```
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input("Enter element: ")))
for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j
    arr[i], arr[min_index] = arr[min_index], arr[i]
print("Sorted list:", arr)
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Large number of elements	1000	Time Error	Sorting may take more time because Selection Sort has $O(n^2)$ time complexity.

Explanation:

1. The program takes the number of elements and then takes each element from the user and stores them in the list arr.

2. The outer for loop selects the current position i and assumes that the element at this position is the smallest.

min_index = i

3. The inner loop compares the selected element with all the elements to its right.

if arr[j] < arr[min_index]:

    min_index = j

4. Whenever a smaller element is found, its index is stored in min_index.

5. After finding the smallest element, it is exchanged with the element at position i.

arr[i], arr[min_index] = arr[min_index], arr[i]

6. The outer loop moves to the next position and repeats the same process until all elements are arranged in ascending order.

7. After sorting is completed, the program displays the sorted list.

WEEK-4

4a) **AIM:** Write a python program to implement merge sort.

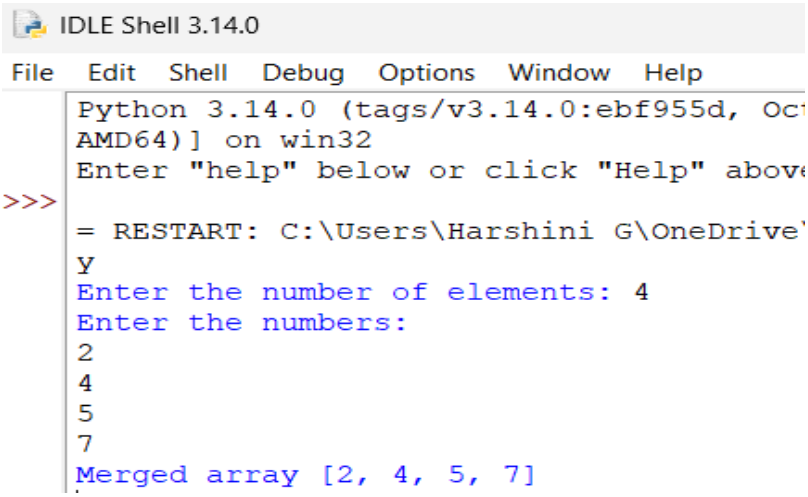
INPUT:

```
def merge_sort(arr):
    #the base condition
    if len(arr)<=1:
```

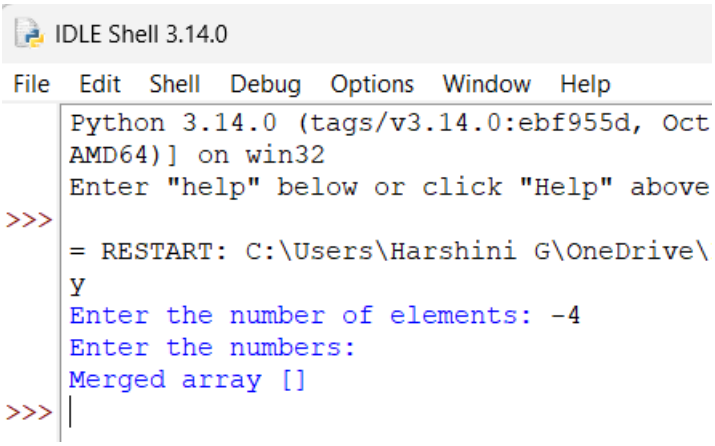
```
    return arr
#dividing the array
mid=len(arr)//2
left=arr[:mid]
right=arr[mid:]
#recurssive calls
merge_sort(left)
merge_sort(right)
i=0
j=0
k=0
#merging the two sorted halves
while i<len(left) and j<len(right):
    if left[i]<=right[j]:
        arr[k]=left[i]
        i+=1
    else:
        arr[k]=right[j]
        j+=1
    k+=1
#copy the remaining elements of left array
while i<len(left):
    arr[k]=left[i]
    i+=1
    k+=1
#copy the remaining elements of right array
while j<len(right):
    arr[k]=right[j]
    j+=1
```

```
        k+=1
#enter the input
n=int(input("Enter the number of elements: "))
arr=[]
print("Enter the numbers:")
for i in range(n):
    arr.append(int(input()))
#function call
merge_sort(arr)
#printing the outputs
print("Merged array",arr)
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Invalid input	a	Type Error	Display an input/type error because int() cannot convert "a"

Explanation:

The program takes the number of elements and stores the input elements in the list arr.

2. The merge_sort() function divides the array into two halves recursively until each half contains one or zero elements.

3. The left and right halves are sorted using recursive calls to merge_sort().

4. The while loop compares elements from both sorted halves and places the smaller element into the original array.

5. The remaining elements of the left and right halves are copied into the array.

6. After merging all the halves, the program displays the sorted array using print("Merged array", arr).

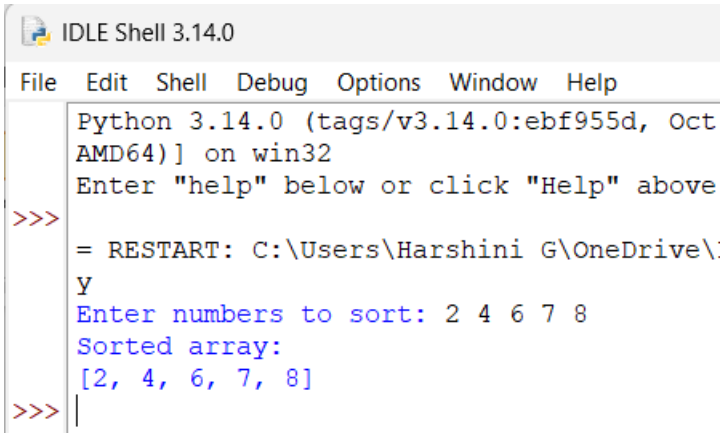
4b) **AIM:** Write a python program to implement quick sort.

INPUT:

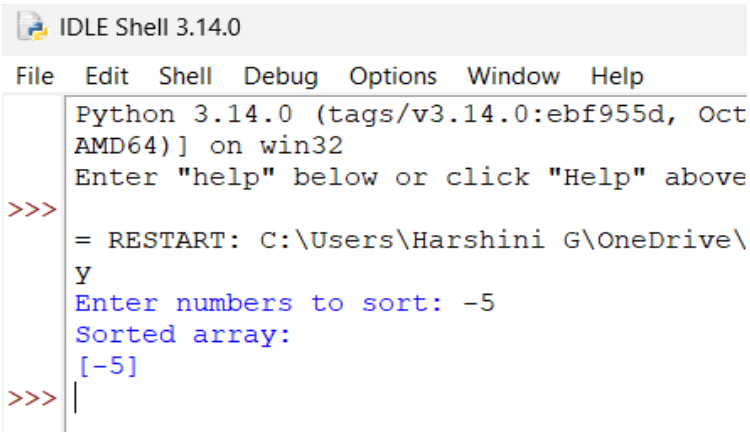
```
def quick_sort(a, low, high):
    #the base condition
    if low < high:
        i = low
        j = high
        pivot = low
        while i < j:
            #element greater than or equal to the pivot
            while i <= high and a[i] <= a[pivot]:
                i += 1
            #element smaller than the pivot
            while a[j] > a[pivot]:
                j -= 1
            if i < j:
```

```
            a[i], a[j] = a[j], a[i]
        #placing the pivots in the positions
        a[pivot], a[j] = a[j], a[pivot]
        #recurrively sortng the left and right parts
        quick_sort(a, low, j - 1)
        quick_sort(a, j + 1, high)
#enter the inputs
a = list(map(int, input("Enter numbers to sort: ").split()))
n = len(a)
#function call
quick_sort(a, 0, n - 1)
#printing the output
print("Sorted array:")
print(a)
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Empty input	-	Will not sort the data and displays	Display Sorted array: [] without performing sorting.

Explanation:

1. The program takes the elements from the user as a list a and finds the number of elements using len(a).
2. The quick_sort() function uses the first element as the pivot and initializes i and j to the low and high positions.
3. The i pointer moves from the left to find an element greater than the pivot, while the j pointer moves from the right to find an element smaller than or equal to the pivot.
4. When i < j, the elements at positions i and j are swapped to arrange them around the pivot.
5. After partitioning, the pivot is swapped with a[j], placing it in its correct position.
6. The function recursively sorts the left and right parts of the array, and finally the program displays the sorted array.

WEEK-5

5a) **AIM:** Write a python program to implement Singly Linked list with the below operation:

i.Create Linked List

ii.Insert beginning

iii.Insert end

iv.Insert at index

v.Delete by value

vi.Delete first node

vii.Delete last node

viii.Count no.of nodes

ix.Exit

INPUT:

```
class Node:
    def __init__(self, data):
        self.data=data
        self.next=None
class SinglyLinkedList:
    def __init__(self):
        self.head=None


    def create(self):
        n=int(input("Enter number of elements:"))
        for i in range(n):
            x=int(input("Enter the value:"))
            new=Node(x)
            if self.head is None:
                self.head=new
            else:
                temp=self.head
                while temp.next:
                    temp=temp.next
                temp.next=new


    def insert_begin(self,data):
        new=Node(data)
        new.next=self.head
        self.head=new
```

```
def insert_end(self,data):
    new=Node(data)
    if self.head is None:
        self.head=new
    else:
        temp=self.head
        while temp.next:
            temp=temp.next
        temp.next=new

def insert_index(self,index,data):
    if index==0:
        self.insert_begin(data)
        return
    elif index>self.count():
        print("Invalid index")
        return
    new=Node(data)
    temp=self.head
    for i in range(index-1):
        temp=temp.next
    new.next=temp.next
    temp.next=new

def delete_Beg(self):
    if self.head is None:
        print("No data to delete")
    else:
```

```
        temp=self.head
        self.head=temp.next
        print("Deleted value=",temp.data)

    def delete_End(self):
        if self.head is None:
            print("No data to delete")
        elif self.head.next is None:
            self.head=None
        else:
            temp=self.head
            temp1=temp
            while temp.next:
                temp1=temp
                temp=temp.next
                temp1.next=None

    def delete(self,value):
        if self.head is None:
            printf("No data to delete")
        else:
            temp=self.head
            if temp and temp.data==value:
                self.head=temp.next
                print("Value deleted")
                return
        while temp.next and temp.next.data!=value:
            temp=temp.next
        if temp.next is None:
```

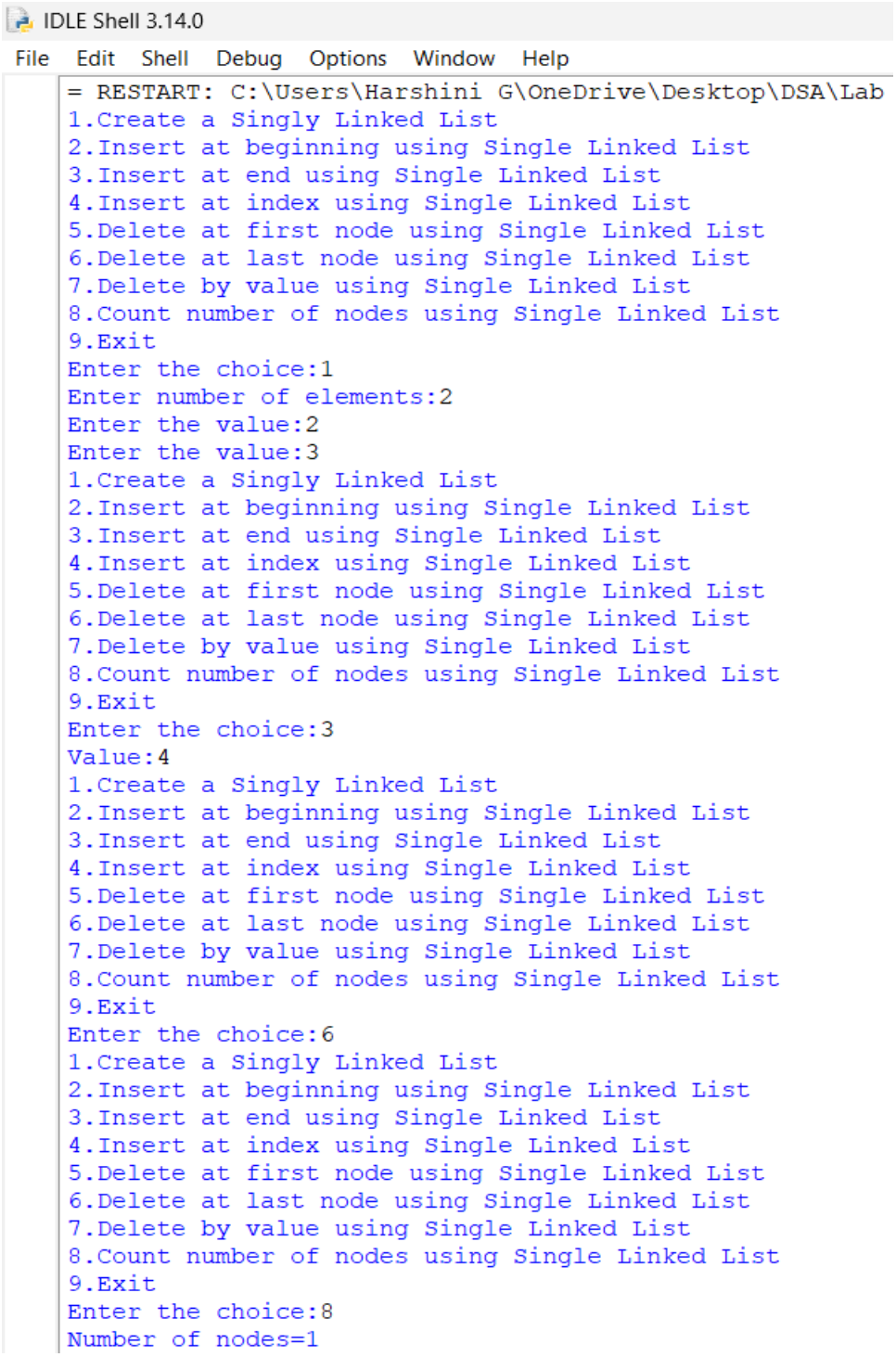
```
            print("Value not present")
        else:
            temp.next=temp.next.next
            print("Value is deleted")

    def count(self):
        if self.head is None:
            print("No linked List")
        else:
            c=0
            temp=self.head
            while temp:
                c=+1
                temp=temp.next
            print(f"Number of nodes={c}")

sll=SinglyLinkedList()
while True:
    print("1.Create a Singly Linked List")
    print("2.Insert at beginning using Single Linked List")
    print("3.Insert at end using Single Linked List")
    print("4.Insert at index using Single Linked List")
    print("5.Delete at first node using Single Linked List")
    print("6.Delete at last node using Single Linked List")
    print("7.Delete by value using Single Linked List")
    print("8.Count number of nodes using Single Linked List")
    print("9.Exit")
    choice=int(input("Enter the choice:"))
    if choice==1:
```

```
        sll.create()
    elif choice==2:
        x=int(input("Value:"))
        sll.insert_begin(x)
    elif choice==3:
        x=int(input("Value:"))
        sll.insert_end(x)
    elif choice==4:
        idx=int(input("Index:"))
        x=int(input("Value:"))
        sll.insert_index(idx,x)
    elif choice==5:
        x=int(input("Delete value:"))
        sll.delete_beg(x)
    elif choice==6:
        sll.delete_end()
    elif choice==7:
        sll.delete()
    elif choice==8:
        sll.count()
    elif choice==9:
        print("The program is exiting")
            break
    else:
        print("Enter a vaild choice")
```

OUTPUT(+VE):



1.Create a Singly Linked List
2.Insert at beginning using Single Linked List
3.Insert at end using Single Linked List
4.Insert at index using Single Linked List
5.Delete at first node using Single Linked List
6.Delete at last node using Single Linked List
7.Delete by value using Single Linked List
8.Count number of nodes using Single Linked List
9.Exit
Enter the choice:9
The program is exiting

OUTPUT(-VE):



```
IDLE Shell 3.14.0
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Harshini G\OneDrive\Desktop\DSA\Lab Manual\Lab5\Single LL.py
1.Create a Singly Linked List
2.Insert at beginning using Single Linked List
3.Insert at end using Single Linked List
4.Insert at index using Single Linked List
5.Delete at first node using Single Linked List
6.Delete at last node using Single Linked List
7.Delete by value using Single Linked List
8.Count number of nodes using Single Linked List
9.Exit
Enter the choice:1
Enter number of elements:1
Enter the value:2
1.Create a Singly Linked List
2.Insert at beginning using Single Linked List
3.Insert at end using Single Linked List
4.Insert at index using Single Linked List
5.Delete at first node using Single Linked List
6.Delete at last node using Single Linked List
7.Delete by value using Single Linked List
8.Count number of nodes using Single Linked List
9.Exit
Enter the choice:2
Value:4
1.Create a Singly Linked List
2.Insert at beginning using Single Linked List
3.Insert at end using Single Linked List
4.Insert at index using Single Linked List
5.Delete at first node using Single Linked List
6.Delete at last node using Single Linked List
7.Delete by value using Single Linked List
8.Count number of nodes using Single Linked List
9.Exit
Enter the choice:4
Index:-2
Value:-2
Number of nodes=1
Traceback (most recent call last):
  File "C:\Users\Harshini G\OneDrive\Desktop\DSA\Lab Manual\Lab5\Single LL.py", line 123, in <module>
    sll.insert_index(idx,x)
  File "C:\Users\Harshini G\OneDrive\Desktop\DSA\Lab Manual\Lab5\Single LL.py", line 41, in insert_index
    elif index>self.count():
TypeError: '>' not supported between instances of 'int' and 'NoneType'
```

Error Table:

Edge Case	Input	Error	Explanation
Count Incorrect	Using c =+1	Count Error	Count remains incorrect; it should be c += 1.

Explanation:

1. The Node class creates each node with data and a next pointer, while SinglyLinkedList maintains the head node.
2. create() builds the linked list, and insert_begin(), insert_end(), and insert_index() add nodes at different positions.
3. delete_Beg(), delete_End(), and delete() remove nodes from the beginning, end, or by a given value.
4. count() traverses the linked list to find and display the total number of nodes.

5. The while loop provides a menu to perform creation, insertion, deletion, counting, or exit operations.
6. **Note:** The code has some errors, such as c=+1 (should be c+=1), incorrect method names in the menu, and missing arguments in delete().

5b) **AIM:** Write a python program to implement a Doubly Linked list with the below operations:

i.Insert beginning

ii.Insert end

iii.Insert at index

iv.Delete by value

v.Delete first node

vi.Delete last node

vii.Count no.of nodes

viii.Exit

INPUT:

```
class Node:
    def __init__(self, data):
        self.prev = None
        self.data = data
        self.next = None



class DoublyLinkedList:
    def __init__(self):
        self.head = None


    def insert_begin(self, data):
        new = Node(data)


        if self.head is None:
```

```
            self.head = new
        else:
            new.next = self.head
            self.head.prev = new
            self.head = new

        print("Insertion completed")

    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head

            while temp.next:
                temp = temp.next

            temp.next = new
            new.prev = temp

        print("Insertion completed")

    def insert_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return
```

```
if index == 0:
    self.insert_begin(data)
    return

if self.head is None:
    print("Invalid index")
    return

temp = self.head

for i in range(index - 1):
    if temp is None:
        print("Invalid index")
        return

    temp = temp.next

if temp is None:
    print("Invalid index")
    return

new = Node(data)

new.next = temp.next
new.prev = temp

if temp.next is not None:
    temp.next.prev = new
```

```
        temp.next = new

        print("Insertion completed")

    def deleteBeg(self):
        if self.head is None:
            print("Can't perform delete operation")
            return

        temp = self.head
        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("Value deleted =", temp.data)

    def deleteEnd(self):
        if self.head is None:
            print("Can't perform delete operation")
            return

        temp = self.head

        if temp.next is None:
            self.head = None
            print("Value deleted =", temp.data)
            return
```

```
    while temp.next:
        temp = temp.next

    print("Value deleted =", temp.data)

    temp.prev.next = None
    temp.prev = None

def deleteIndex(self, index):
    if self.head is None:
        print("Can't perform delete operation")
        return

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.deleteBeg()
        return

    temp = self.head

    for i in range(index):
        if temp is None:
            print("Invalid index")
            return

        temp = temp.next
```

```
        if temp is None:
            print("Invalid index")
            return

        if temp.prev is not None:
            temp.prev.next = temp.next

        if temp.next is not None:
            temp.next.prev = temp.prev

        print("Value deleted =", temp.data)

    def count(self):
        c = 0
        temp = self.head

        while temp:
            c += 1
            temp = temp.next

        print(f"Number of nodes = {c}")

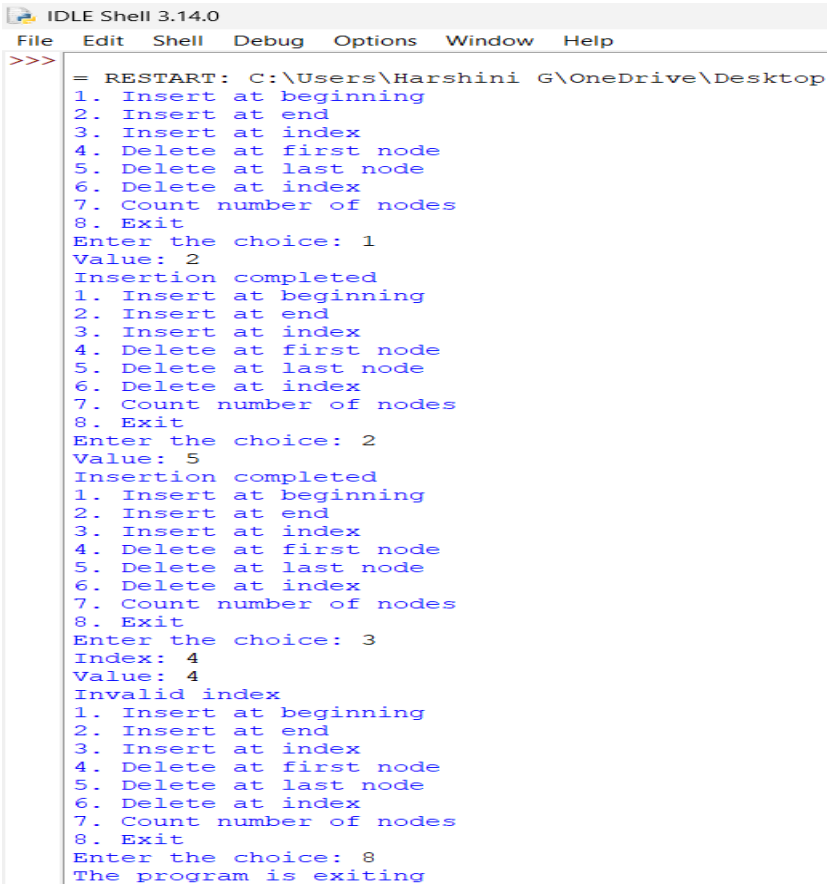

dll = DoublyLinkedList()

while True:
    print("1. Insert at beginning")
    print("2. Insert at end")
```

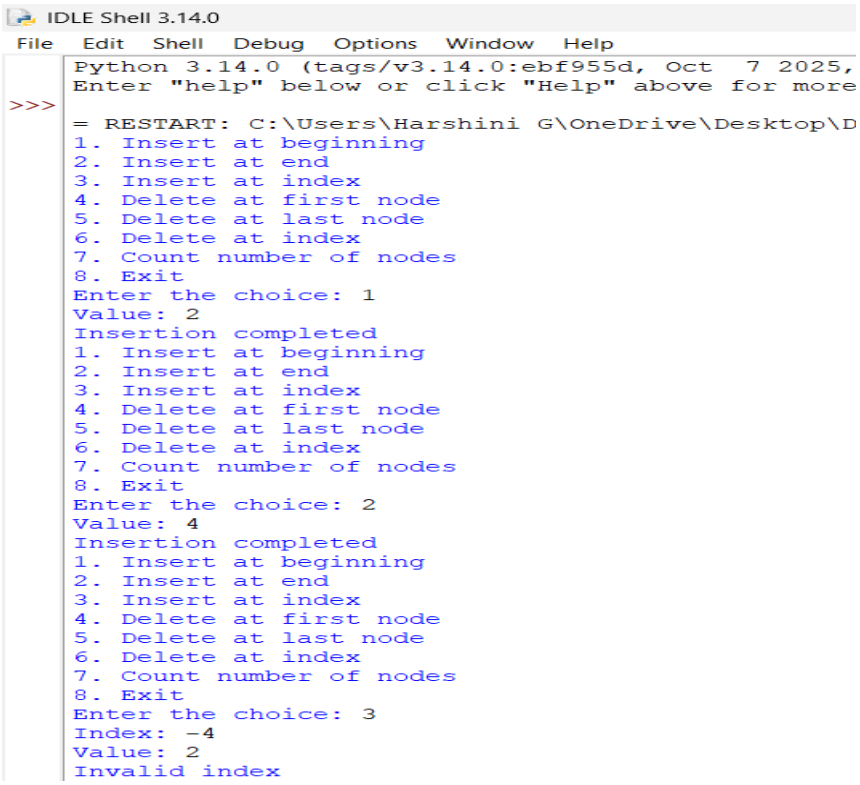
```
print("3. Insert at index")
print("4. Delete at first node")
print("5. Delete at last node")
print("6. Delete at index")
print("7. Count number of nodes")
print("8. Exit")
choice = int(input("Enter the choice: "))
if choice == 1:
    x = int(input("Value: "))
    dll.insert_begin(x)
elif choice == 2:
    x = int(input("Value: "))
    dll.insert_end(x)
elif choice == 3:
    idx = int(input("Index: "))
    x = int(input("Value: "))
    dll.insert_index(idx, x)
elif choice == 4:
    dll.deleteBeg()
elif choice == 5:
    dll.deleteEnd()
elif choice == 6:
    idx = int(input("Delete index: "))
    dll.deleteIndex(idx)
elif choice == 7:
    dll.count()
elif choice == 8:
    print("The program is exiting")
    break
```

```
    else:
        print("Enter a valid choice")
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Invalid input type	abc	Input Error	Display an input error because the value must be an integer.

Explanation:

1. The Node class represents each node with data, prev, and next pointers for a doubly linked list.
2. insert_begin(), insert_end(), and insert_index() add new nodes at the beginning, end, or a specified index.
3. deleteBeg(), deleteEnd(), and deleteIndex() remove nodes from the beginning, end, or a specified index.
4. The count() function traverses the list and counts the total number of nodes.
5. The head pointer stores the address of the first node, while prev and next connect nodes in both directions.
6. The while loop provides a menu to perform insertion, deletion, counting, or exit operations.

5c) **AIM:** Write a python program to implement a Circulated Linked list with the given operations:

i.Insert beginning

ii.Insert end

iii.Insert at index

iv.Delete by value

v.Delete first node

vi.Delete last node

vii.Count no.of nodes

viii.Exit

INPUT:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def insert_begin(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            self.tail = new
            new.next = self.head
        else:
            new.next = self.head
            self.head = new
            self.tail.next = self.head
```

```
    print("Insertion completed")

def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
        self.tail = new
        new.next = self.head
    else:
        new.next = self.head
        self.tail.next = new
        self.tail = new

    print("Insertion completed")

def insert_position(self, position, data):
    if position < 0:
        print("Invalid position")
        return

    if position == 0:
        self.insert_begin(data)
        return

    if self.head is None:
        print("Invalid position")
        return
```

```
        temp = self.head

        for i in range(position - 1):
            temp = temp.next

            if temp == self.head:
                print("Invalid position")
                return

        new = Node(data)
        new.next = temp.next
        temp.next = new

        if temp == self.tail:
            self.tail = new

        print("Insertion completed")

    def delete_begin(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head == self.tail:
            print("Deleted value =", self.head.data)
            self.head = None
            self.tail = None
        else:
            print("Deleted value =", self.head.data)
```

```
            self.head = self.head.next
            self.tail.next = self.head

    def delete_end(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head == self.tail:
            print("Deleted value =", self.tail.data)
            self.head = None
            self.tail = None
            return

        temp = self.head

        while temp.next != self.tail:
            temp = temp.next

        print("Deleted value =", self.tail.data)

        temp.next = self.head
        self.tail = temp

    def delete_position(self, position):
        if self.head is None:
            print("List is empty")
            return
```

```
if position < 0:
    print("Invalid position")
    return

if position == 0:
    self.delete_begin()
    return

temp = self.head

for i in range(position - 1):
    temp = temp.next

    if temp == self.head:
        print("Invalid position")
        return

if temp.next == self.head:
    print("Invalid position")
    return

delete_node = temp.next

if delete_node == self.tail:
    self.tail = temp

temp.next = delete_node.next

print("Deleted value =", delete_node.data)
```

```
    def count(self):
        if self.head is None:
            print("Number of nodes = 0")
            return

        c = 0
        temp = self.head

        while True:
            c += 1
            temp = temp.next

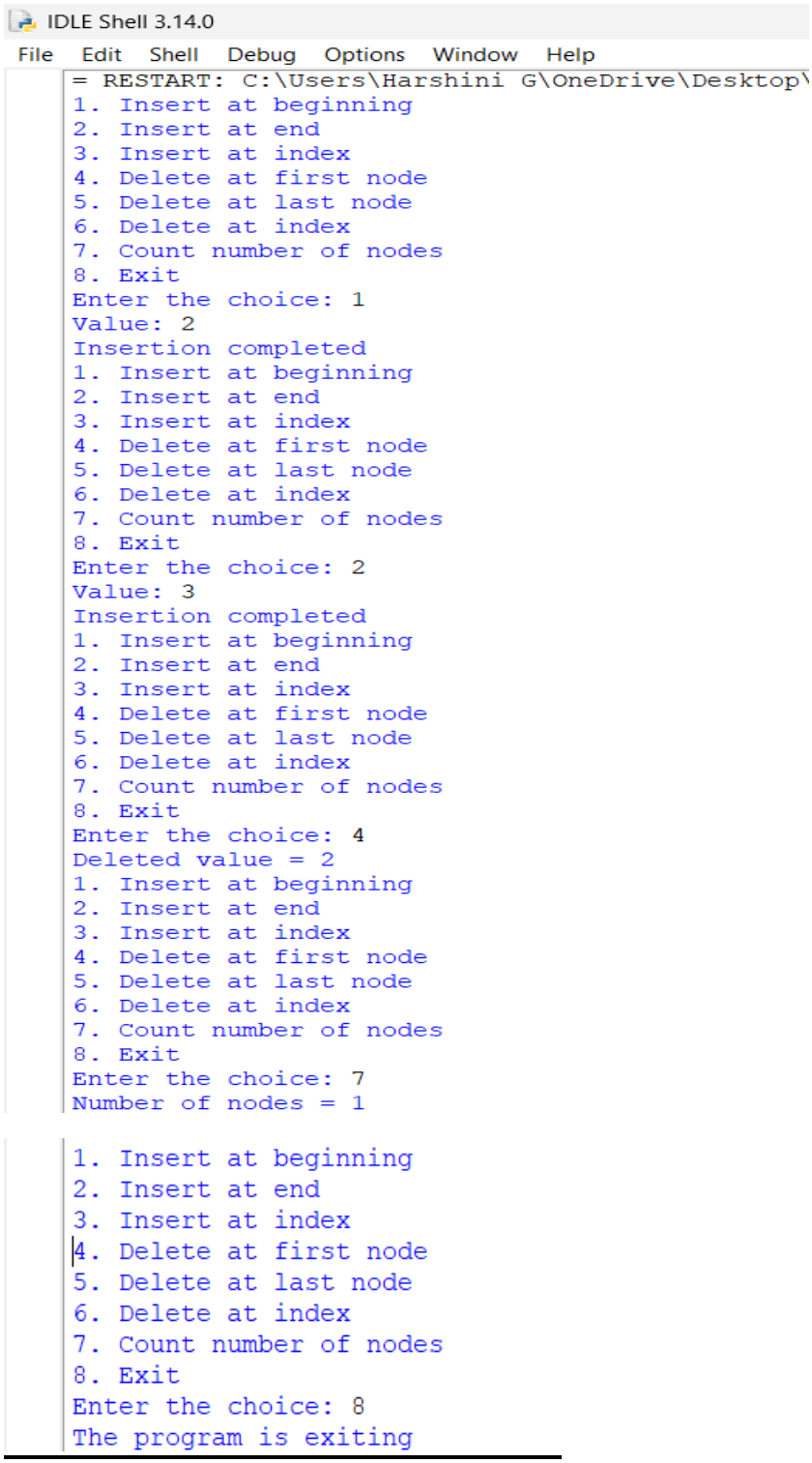
            if temp == self.head:
                break

        print("Number of nodes =", c)

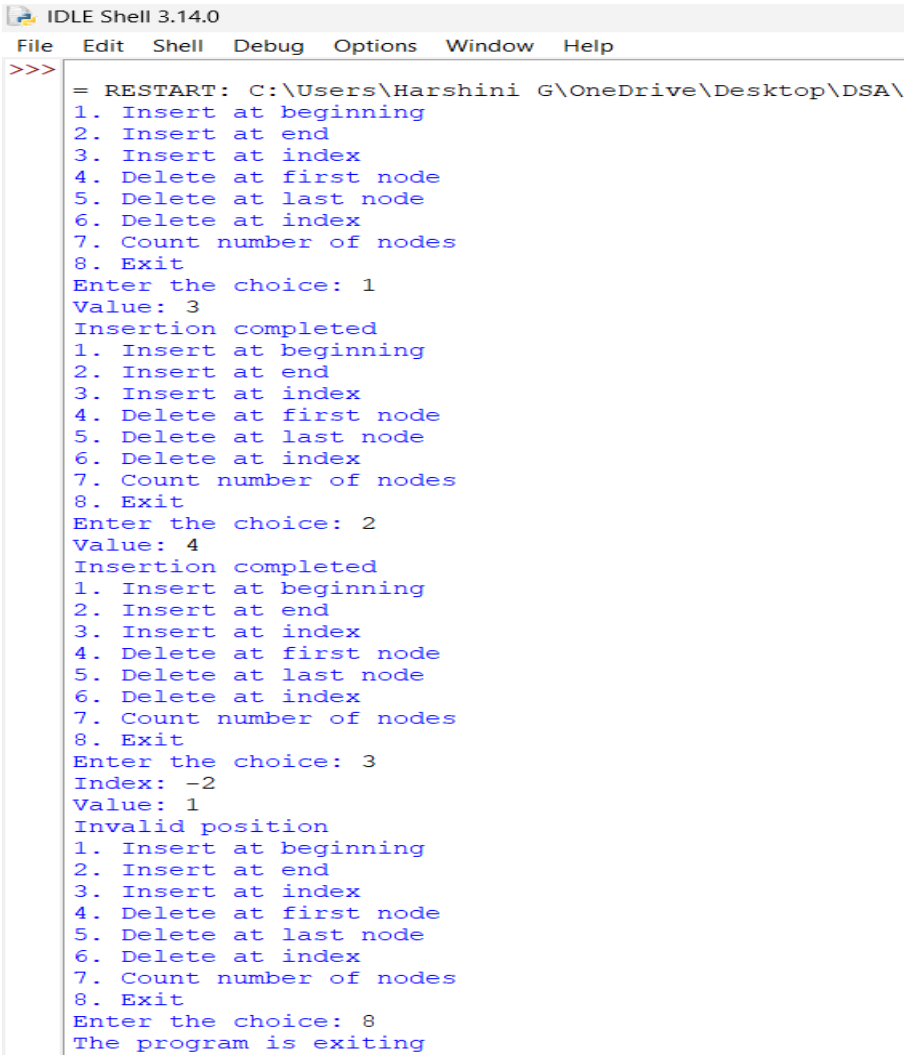
cll = CircularLinkedList()
while True:
    print("1. Insert at beginning")
    print("2. Insert at end")
    print("3. Insert at index")
    print("4. Delete at first node")
    print("5. Delete at last node")
    print("6. Delete at index")
    print("7. Count number of nodes")
    print("8. Exit")
    choice = int(input("Enter the choice: "))
```

```
    if choice == 1:
        x = int(input("Value: "))
        cll.insert_begin(x)
    elif choice == 2:
        x = int(input("Value: "))
        cll.insert_end(x)
    elif choice == 3:
        idx = int(input("Index: "))
        x = int(input("Value: "))
        cll.insert_position(idx, x)
    elif choice == 4:
        cll.delete_begin()
    elif choice == 5:
        cll.delete_end()
    elif choice == 6:
        idx = int(input("Delete index: "))
        cll.delete_position(idx)
    elif choice == 7:
        cll.count()
    elif choice == 8:
        print("The program is exiting")
        break
    else:
        print("Enter a valid choice")
```

OUTPUT(+VE):


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
= RESTART: C:\Users\Harshini G\OneDrive\Desktop\
1. Insert at beginning
2. Insert at end
3. Insert at index
4. Delete at first node
5. Delete at last node
6. Delete at index
7. Count number of nodes
8. Exit
Enter the choice: 1
Value: 2
Insertion completed
1. Insert at beginning
2. Insert at end
3. Insert at index
4. Delete at first node
5. Delete at last node
6. Delete at index
7. Count number of nodes
8. Exit
Enter the choice: 2
Value: 3
Insertion completed
1. Insert at beginning
2. Insert at end
3. Insert at index
4. Delete at first node
5. Delete at last node
6. Delete at index
7. Count number of nodes
8. Exit
Enter the choice: 4
Deleted value = 2
1. Insert at beginning
2. Insert at end
3. Insert at index
4. Delete at first node
5. Delete at last node
6. Delete at index
7. Count number of nodes
8. Exit
Enter the choice: 7
Number of nodes = 1
1. Insert at beginning
2. Insert at end
3. Insert at index
4. Delete at first node
5. Delete at last node
6. Delete at index
7. Count number of nodes
8. Exit
Enter the choice: 8
The program is exiting

OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Invalid menu choice	10	Invalid value	Display "Enter a valid choice".

Explanation:

1.The program creates nodes containing data and a next pointer, with tail.next always pointing to head to maintain the circular structure.

2. insert_begin(), insert_end(), and insert_position() add new nodes at the beginning, end, or specified position.

3. delete_begin(), delete_end(), and delete_position() remove nodes from the beginning, end, or specified position.

4. The head and tail pointers are updated after every insertion or deletion to maintain the circular connection.

5. The count() and display() functions traverse the list and stop when the pointer reaches head again.

6. The menu-driven program allows the user to perform these operations repeatedly until the Exit option is selected.

WEEK-6

6a) **AIM:** Write a python program to implement stack operations such as push(),pop(),peek().Display() using an array.

INPUT:

```
class stack:
    def __init__(self,size):
        self.size=size
        self.stack=[None]*size
        self.top=-1

    def push(self,data):
        if self.top==self.size-1:
            print("Stack Overflow")
        else:
            self.top+=1
            self.stack[self.top]=item
            print(item,"pushed into the stack")

    def pop(self):
        if self.top is None:
            print("Stack underflow")
        else:
            item=self.stack[self.top]
            self.stack[self.top]=None
            self.top-=1
            print(item,"popped from the stack")
```

```
    def peek(self):
        if self.top==-1:
            print("Stack is empty")
        else:
            print("Top element:",self.stack[self.top])

    def display(self):
        if self.top==-1:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            for i in range(self.top,-1,-1):
                print(self.stack[i])

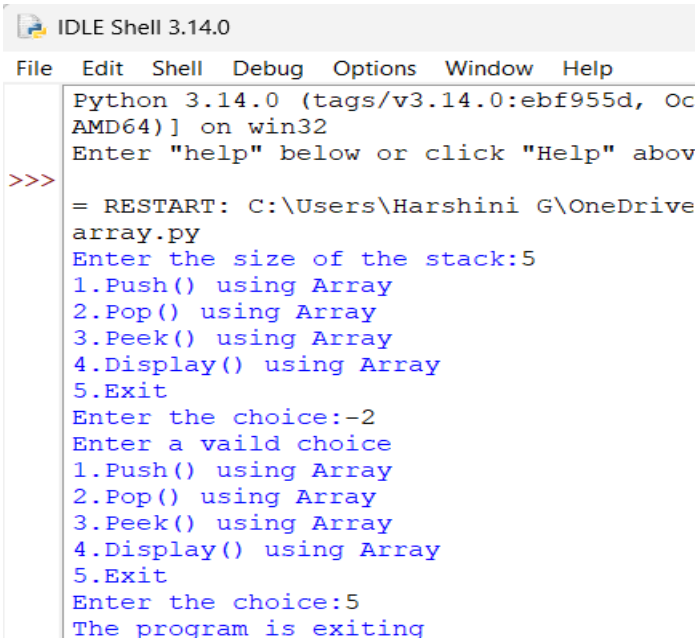
size=int(input("Enter the size of the stack:"))
s=stack(size)
while True:
    print("1.Push() using Array")
    print("2.Pop() using Array")
    print("3.Peek() using Array")
    print("4.Display() using Array")
    print("5.Exit")
    choice=int(input("Enter the choice:"))
    if choice==1:
        item=int(input("Enter the element to push:"))
        s.push(item)
    elif choice==2:
        s.pop()
    elif choice==3:
```

```
        s.peek()
    elif choice==4:
        s.display()
    elif choice==5:
        print("The program is exiting")
        break
    else:
        print("Enter a vaild choice")
```

OUTPUT(+VE):



OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Invalid stack size	0	Invalid input	Display an "Invalid size" message.

Explanation:

1. The stack class implements a stack using an **array (list)** with a fixed size and top keeps track of the top element.
2. push() adds an element to the stack and displays **Stack Overflow** when the stack is full.
3. pop() removes the top element and displays **Stack Underflow** when the stack is empty.
4. peek() displays the current top element without removing it from the stack.
5. display() prints all the stack elements from top to bottom.
6. The while loop provides a menu to perform Push, Pop, Peek, Display, or Exit operations.

6b) **AIM:** Write a python program to implement stack operations such as push(),pop(),peek().Display() using an Linked list.

INPUT:

```
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None
```

```
class stack:
    def __init__(self):
        self.top=None

    def push(self,data):
        new=Node(data)
        new.next=self.top
        self.top=new
        print(data,"pushed into stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp=self.top
            print(temp.data,"popped from stack")
            self.top=self.top.next

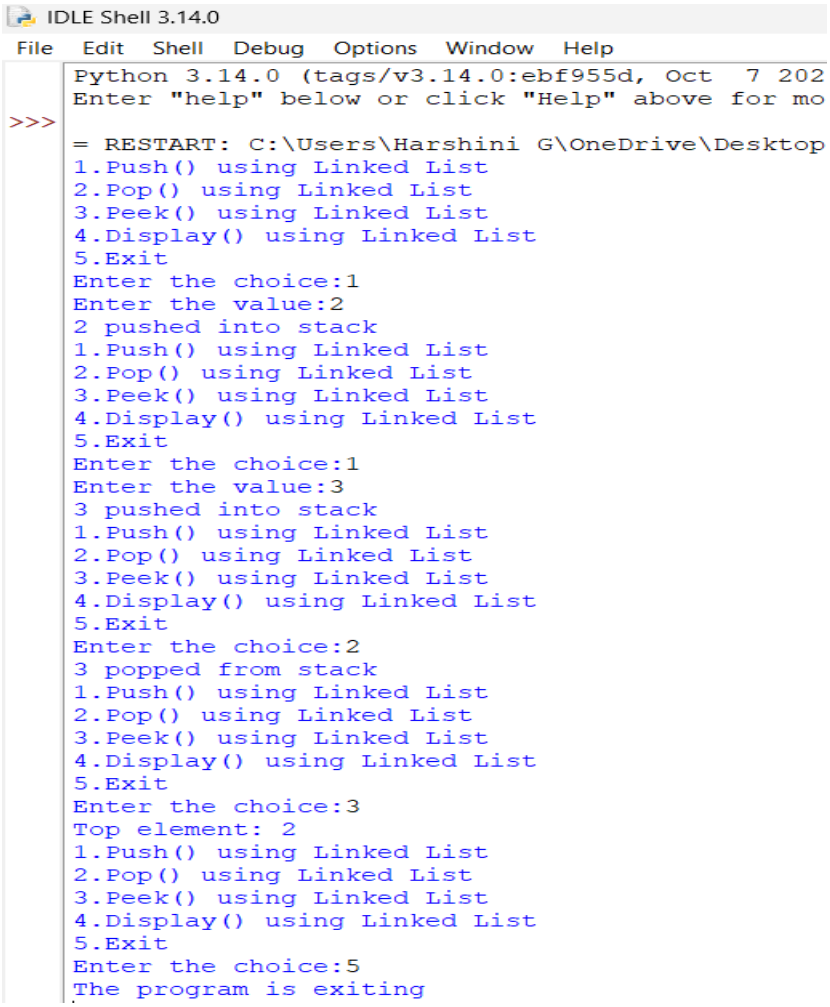
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:",self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp=self.top
```

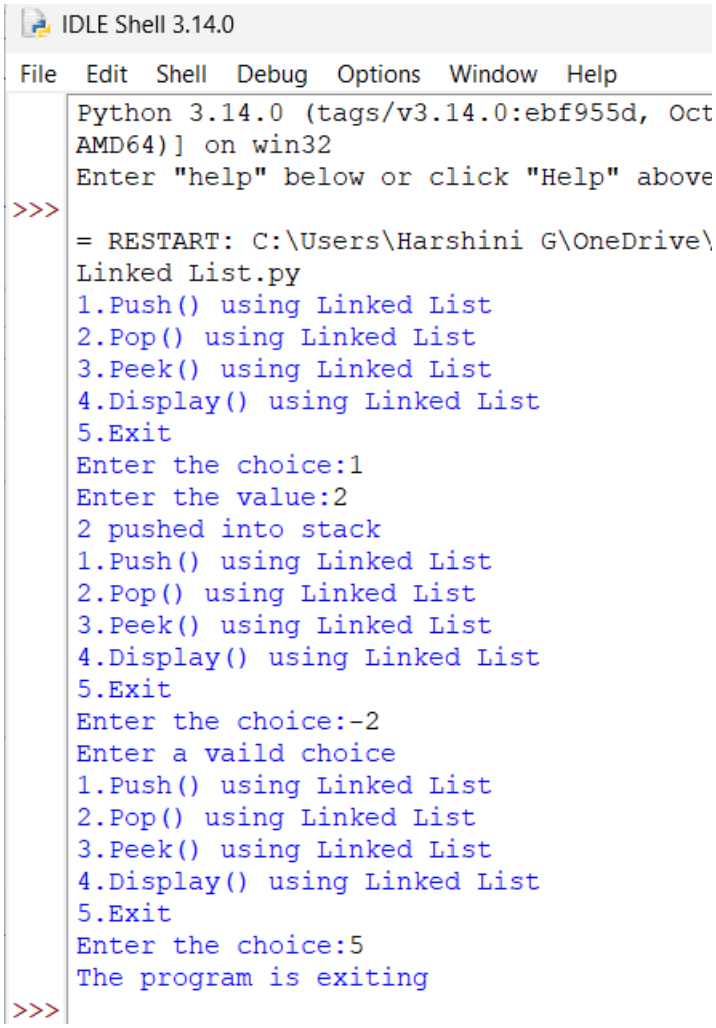
```
            print("Stack elements")
            while temp is not None:
                print(temp.data)
                temp=temp.next


s=stack()
while True:
    print("1.Push() using Linked List")
    print("2.Pop() using Linked List")
    print("3.Peek() using Linked List")
    print("4.Display() using Linked List")
    print("5.Exit")
    choice=int(input("Enter the choice:"))
    if choice==1:
        data=int(input("Enter the value:"))
        s.push(data)
    elif choice==2:
        s.pop()
    elif choice==3:
        s.peek()
    elif choice==4:
        s.display()
    elif choice==5:
        print("The program is exiting")
        break
    else:
        print("Enter a vaild choice")
```

OUTPUT(+VE):


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 202
Enter "help" below or click "Help" above for mo
>>>
= RESTART: C:\Users\Harshini G\OneDrive\Desktop
1.Push() using Linked List
2.Pop() using Linked List
3.Peek() using Linked List
4.Display() using Linked List
5.Exit
Enter the choice:1
Enter the value:2
2 pushed into stack
1.Push() using Linked List
2.Pop() using Linked List
3.Peek() using Linked List
4.Display() using Linked List
5.Exit
Enter the choice:1
Enter the value:3
3 pushed into stack
1.Push() using Linked List
2.Pop() using Linked List
3.Peek() using Linked List
4.Display() using Linked List
5.Exit
Enter the choice:2
3 popped from stack
1.Push() using Linked List
2.Pop() using Linked List
3.Peek() using Linked List
4.Display() using Linked List
5.Exit
Enter the choice:3
Top element: 2
1.Push() using Linked List
2.Pop() using Linked List
3.Peek() using Linked List
4.Display() using Linked List
5.Exit
Enter the choice:5
The program is exiting

OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Invalid input type	abc	Input Error	Display an input error because the stack expects an integer.

Explanation:

1. Node creates each element of the linked list with data and a next pointer.
2. The stack class uses top to keep track of the top element of the stack.
3. push() adds a new node at the top, following the **LIFO (Last In, First Out)** principle.
4. pop() removes the top node and displays **Stack Underflow** if the stack is empty.
5. peek() shows the top element, while display() prints all stack elements from top to bottom.
6. The while loop provides a menu to perform Push, Pop, Peek, Display, or Exit operations.

WEEK-7

7a) **AIM:** Write a python program to implement a queue and its operations such as enqueue,dequeue,peek and display with a menu driven approach.Implement using array.

INPUT:

```
class Queue:
    def __init__(self,size):
        self.size=size
        self.queue=[None]*size
        self.front=-1
        self.rear=-1

    def enqueue(self,x):
        if self.rear==self.size-1:
            print("Queue Overflow")
        else:
            if self.front==-1:
                self.front=0
            self.rear+=1
            self.queue[self.rear]=x
            print(f"{x} inserted into queue")

    def dequeue(self):
        if self.front==-1 or self.front>self.rear:
            print("Queue Underflow")
        else:
            x=self.queue[self.front]
            self.queue[self.front]=None
            self.front+=1
            print(f"{x} deleted from the queue")
```

```
            if self.front>self.rear:
                self.front= -1
                self.rear= -1

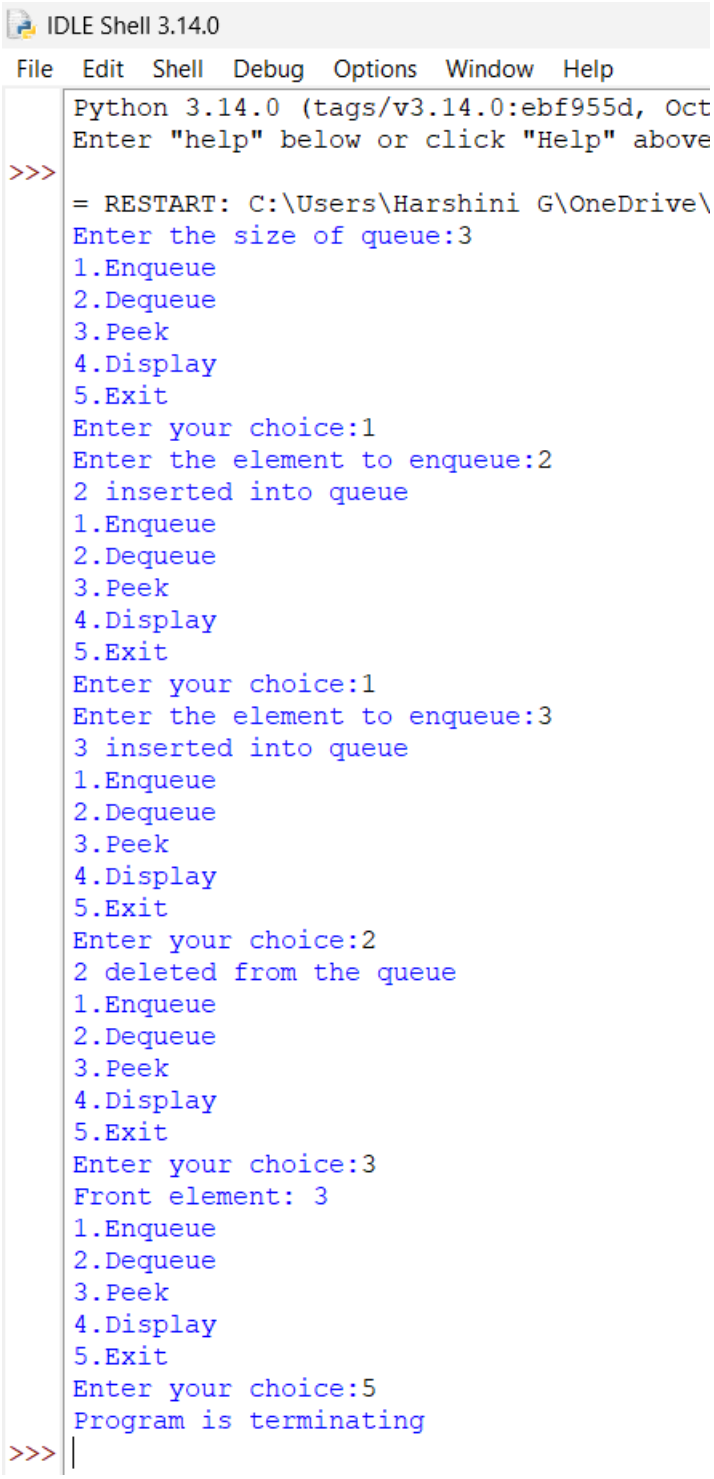
    def peek(self):
        if self.front==-1:
            print("Queue is empty")
        else:
            print("Front element:",self.queue[self.front])

    def display(self):
        if self.front==-1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front,self.rear+1):
                print(self.queue[i])

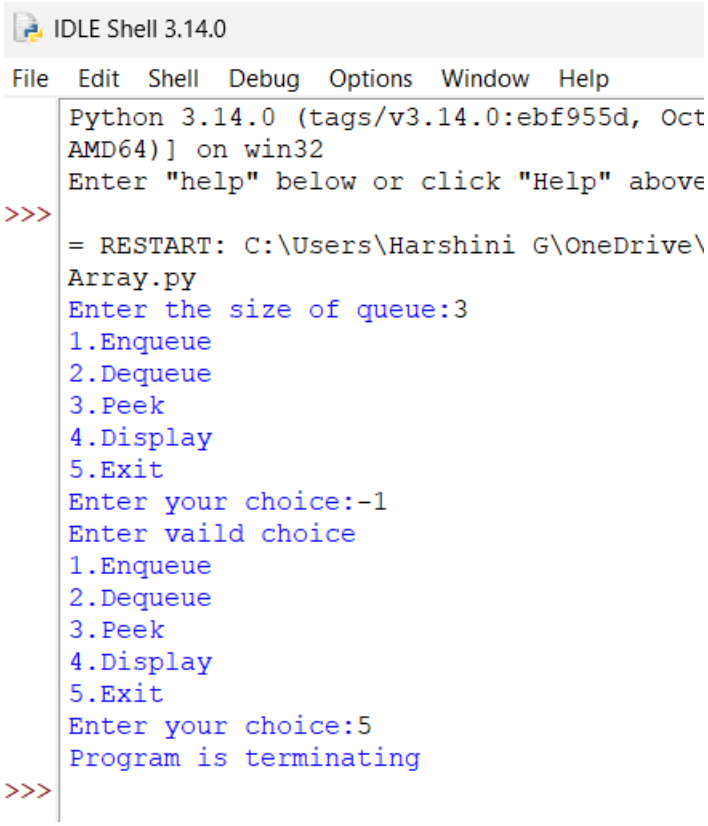
size=int(input("Enter the size of queue:"))
q=Queue(size)
while True:
    print("1.Enqueue")
    print("2.Dequeue")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")
    choice=int(input("Enter your choice:"))
    if choice==1:
        x=int(input("Enter the element to enqueue:"))
```

```
        q.enqueue(x)
    elif choice==2:
        q.dequeue()
    elif choice==3:
        q.peek()
    elif choice==4:
        q.display()
    elif choice==5:
        print("Program is terminating")
        break
    else:
        print("Enter vaild choice")
```

OUTPUT(+VE):


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct
Enter "help" below or click "Help" above
>>>
= RESTART: C:\Users\Harshini G\OneDrive\
Enter the size of queue:3
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:1
Enter the element to enqueue:2
2 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:1
Enter the element to enqueue:3
3 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:2
2 deleted from the queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:3
Front element: 3
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:5
Program is terminating
>>>

OUTPUT(-VE):



Error table:

Edge Case	Input	Error	Explanation
Invalid queue size	0	Input error	Display an "Invalid size" message.

Explanation:

1. The program creates a queue using an array, with front and rear variables to track the first and last elements.
2. The enqueue() function inserts an element at the rear and displays **Queue Overflow** when the array is full.
3. The dequeue() function removes the element from the front and displays **Queue Underflow** when the queue is empty.
4. When all elements are deleted, both front and rear are reset to -1.
5. The peek() function displays the front element, while display() prints all elements from front to rear.
6. The menu-driven program allows enqueue, dequeue, peek, and display operations until the user selects **Exit**.

7b) **AIM:** Write a python program to implement a queue and its operations such as enqueue,dequeue,peek and display with a menu driven approach.Implement using Singly Linked list.

INPUT:

```
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None


class Queue:
    def __init__ (self):
        self.front=None
        self.rear=None


    def enqueue(self,x):
        new=Node(x)
        if self.rear is None:
            self.front=self.rear=new
        else:
            self.rear.next=new
            self.rear=new
            print(x,"inserted into the queue.")


    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            x=self.front.data
            self.front=self.front.next
```

```
            if self.front is None:
                self.rear=None
            print(f"{x} deleted from the queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:",self.front.data)

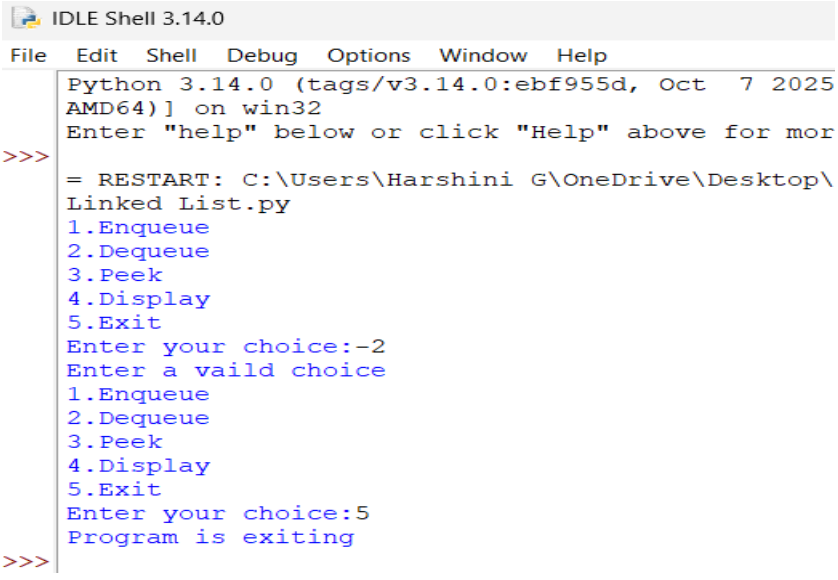
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            temp=self.front
            while temp is not None:
                print(temp.data)
                temp=temp.next
q=Queue()
while True:
    print("1.Enqueue")
    print("2.Dequeue")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")
    choice=int(input("Enter your choice:"))
    if choice==1:
        x=int(input("Enter the element to enqueue:"))
```

```
        q.enqueue(x)
    elif choice==2:
        q.dequeue()
    elif choice==3:
        q.peek()
    elif choice==4:
        q.display()
    elif choice==5:
        print("Program is exiting")
        break
    else:
        print("Enter a vaild choice")
```

OUTPUT(+VE):


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025,
Enter "help" below or click "Help" above for mor
>>>
= RESTART: C:\Users\Harshini G\OneDrive\Desktop\
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:1
Enter the element to enqueue:2
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:1
Enter the element to enqueue:3
3 inserted into the queue.
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:2
2 deleted from the queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:3
Front element: 3
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:5
Program is exiting
>>>

OUTPUT(-VE):


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025
AMD64)] on win32
Enter "help" below or click "Help" above for mor
>>>
= RESTART: C:\Users\Harshini G\OneDrive\Desktop\
Linked List.py
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:-2
Enter a vaild choice
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice:5
Program is exiting
>>>

Error table:

Edge Case	Input	Error	Explanation
Invalid menu choice	10	Input Error	Display "Enter a valid choice".

Explanation:

1.The program creates a Node class with data and next to store elements and link them together.

2. The Queue class uses front and rear pointers to keep track of the first and last nodes.

3. The enqueue() function creates a new node and adds it at the rear of the queue.

4. The dequeue() function removes the node from the front and displays **Queue Underflow** if the queue is empty.

5. The peek() function displays the front element, while display() traverses the linked list and prints all queue elements.

6. The menu-driven program allows the user to perform enqueue, dequeue, peek, and display operations until **Exit** is selected.

7c) **AIM:** Write a python program to implement a queue and its operations such as enqueue,dequeue,peek and display with a menu driven approach.Implement using Circulated queue.

INPUT:

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
```

```
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        elif self.front == self.rear:  # Only one element
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")
        else:  # More than one element
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
            print(item, "deleted from the queue")

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
```

```
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
            return None
        else:
            print("Front element is:", self.queue[self.front])
            return self.queue[self.front]

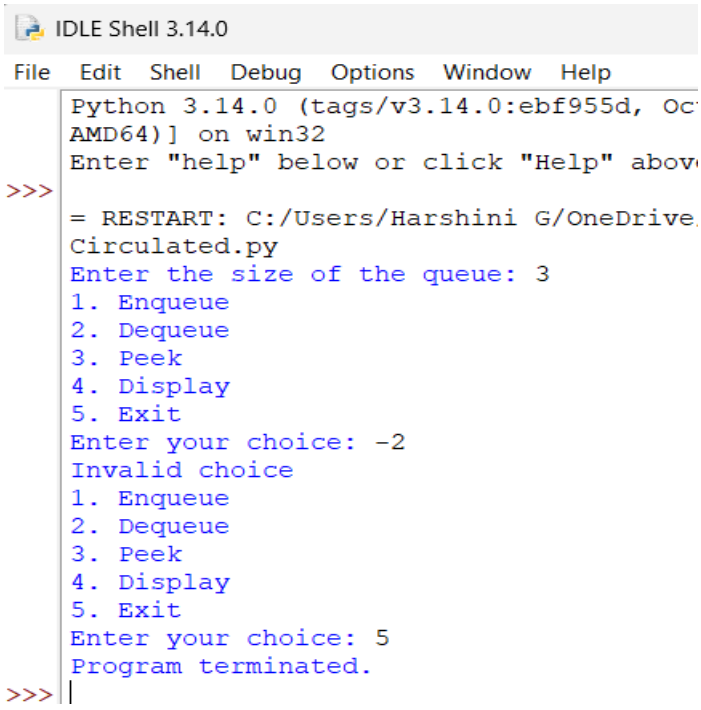
# Menu-driven program
size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)
while True:
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
```

```
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT(+VE):



OUTPUT(-VE):



Error table:

Edge Case	Input	Error	Explanation
Invalid input type	abc	Input Error	Display an input error because the queue expects integer values.

Explanation:

1. The program creates a circular queue using an array, with front and rear pointers to track the first and last elements.
2. The enqueue() function inserts an element and uses modulo % size to move rear circularly.
3. If the next position of rear is equal to front, the queue is full and **Queue Overflow** is displayed.
4. The dequeue() function removes the front element and updates front; if the queue is empty, **Queue Underflow** is displayed.
5. The peek() function displays the front element, while display() prints all elements by traversing circularly from front to rear.
6. The menu-driven program allows the user to perform enqueue, dequeue, peek, and display operations until **Exit** is selected.

WEEK-8

8a) **AIM:** Write a python program to implement a binary tree with its operations using array.

INPUT:

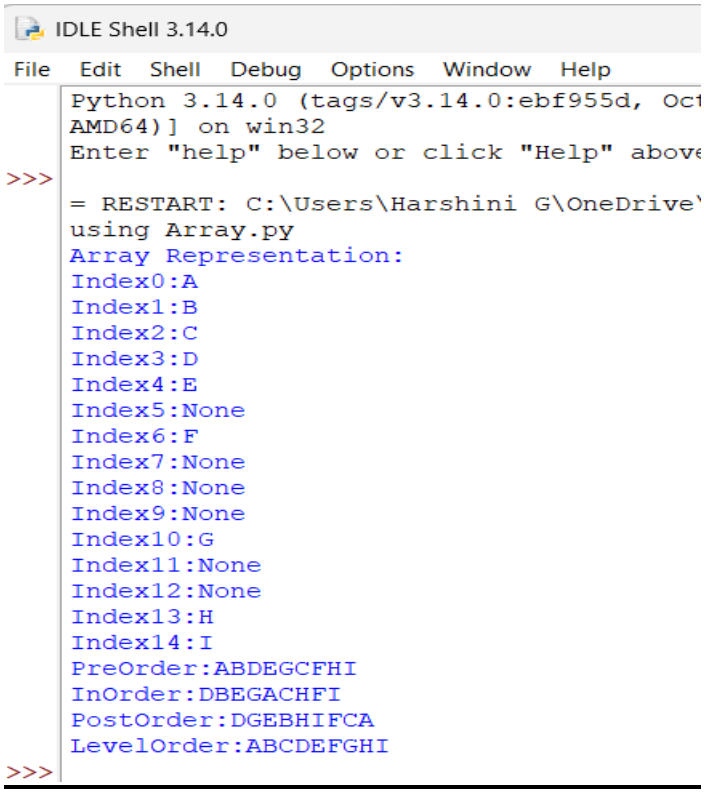
```
class BinaryTreeArray:
    def __init__(self,size):
        self.tree=[None]*size
        self.size=size
#insert root
    def set_root(self,data):
        self.tree[0]=data
#set left child
    def set_left(self,parent_index,data):
        child_index=2*parent_index+1
        if child_index<self.size:
            self.tree[child_index]=data
        else:
            print("Index out of range")
#set right child
    def set_right(self,parent_index,data):
        child_index=2*parent_index+2
        if child_index<self.size:
            self.tree[child_index]=data
        else:
            print("Index out of range")
#preorder Traversal
    def preorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        print(self.tree[index],end="")
        self.preorder(2*index+1)
        self.preorder(2*index+2)
#inorder Traversal
```

```
    def inorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        self.inorder(2*index+1)
        print(self.tree[index],end="")
        self.inorder(2*index+2)
#postorder Traversal
    def postorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        self.postorder(2*index+1)
        self.postorder(2*index+2)
        print(self.tree[index],end="")
#levelorder Traversal
    def levelorder(self):
        for value in self.tree:
            if value is not None:
                print(value,end="")
#display Array
    def display(self):
        for i in range(self.size):
            print(f"Index{i}:{self.tree[i]}")

tree=BinaryTreeArray(15)
tree.set_root("A")
tree.set_left(0,'B')
tree.set_right(0,'C')
tree.set_left(1,'D')
tree.set_right(1,'E')
```

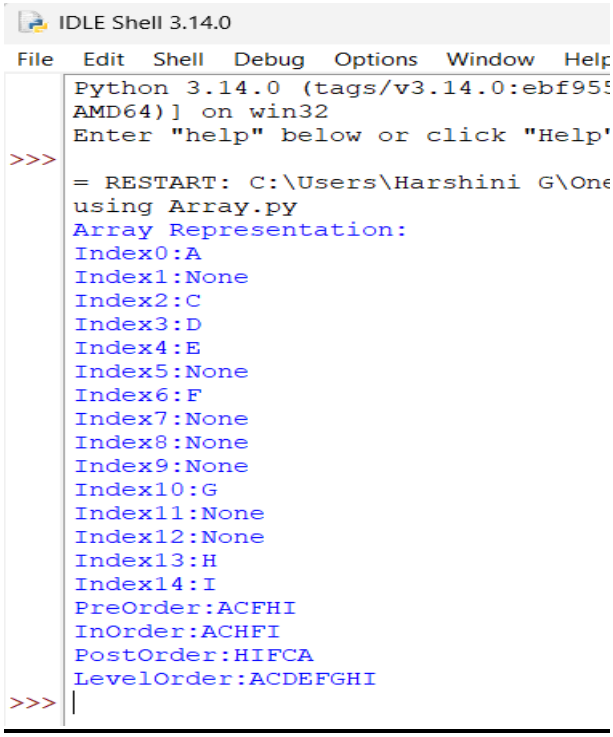
```
tree.set_right(2,'F')
tree.set_right(4,'G')
tree.set_left(6,'H')
tree.set_right(6,'I')
print("Array Representation:")
tree.display()
print("PreOrder:",end="")
tree.preorder()
print("\nInOrder:",end="")
tree.inorder()
print("\nPostOrder:",end="")
tree.postorder()
print("\nLevelOrder:",end="")
tree.levelorder()
```

OUTPUT(+VE):



OUTPUT(-VE):

If tree.set_left(-1, "X") then, the output is:



Error Table:

Edge Case	Input	Error	Explanation
Left child out of range	tree.set_left(7, "X") with size 15	Index out of range	Left child index is 15, outside the array.

Explanation:

1) The program creates a BinaryTreeArray class with a list of a specified size to store tree elements.

2) The set_root() function inserts the root node at index 0.

3) The set_left() function inserts a left child at index 2 * parent_index + 1.

4) The set_right() function inserts a right child at index 2 * parent_index + 2 and prints an error if the index is out of range.

5) The preorder(), inorder(), and postorder() functions perform tree traversals recursively in different orders.

6) The levelorder() function displays the non-empty elements level by level, and display() prints each array index and its value.

7) Finally, the program creates a tree, inserts nodes A to I, and displays the array representation and all four traversals.

8b) **AIM:** Write a python program to implement a binary tree with its operations using linked list.

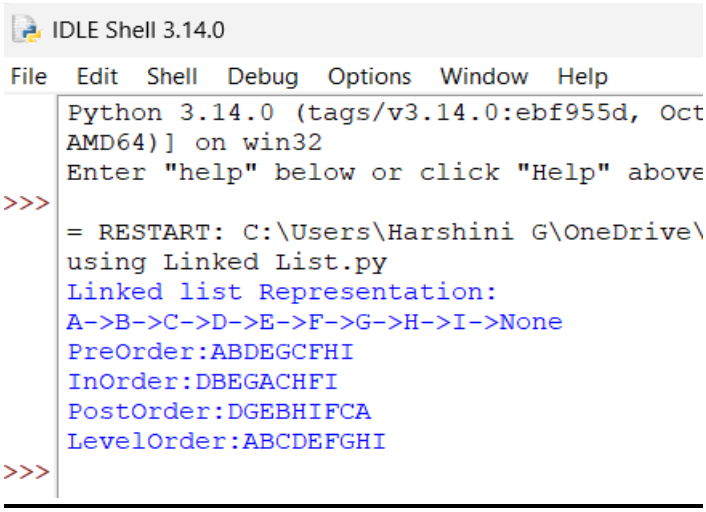
INPUT:

```
class Node:
    def __init__(self,data):
        self.data=data
        self.left=None
        self.right=None
class BinaryTree:
    def __init__(self):
        self.root=None
#Create Tree
    def create_tree(self):
        self.root=Node('A')
        self.root.left=Node('B')
        self.root.right=Node('C')
        self.root.left.left=Node('D')
        self.root.left.right=Node('E')
        self.root.right.right=Node('F')
        self.root.left.right.right=Node('G')
        self.root.right.right.left=Node('H')
        self.root.right.right.right=Node('I')
#Preorder Traversal
    def preorder(self,node):
        if node is None:
            return
        print(node.data,end="")
        self.preorder(node.left)
        self.preorder(node.right)
#Inorder Traversal
    def inorder(self,node):
```

```
        if node is None:
            return
        self.inorder(node.left)
        print(node.data,end="")
        self.inorder(node.right)
    #Postorder Traversal
    def postorder(self,node):
        if node is None:
            return
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data,end="")
    #levelorder Traversal
    def levelorder(self):
        if self.root is None:
            return
        queue=[self.root]
        while queue:
            node=queue.pop(0)
            print(node.data,end="")
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)
    #display
    def display(self):
        if self.root is None:
            return
        queue=[self.root]
```

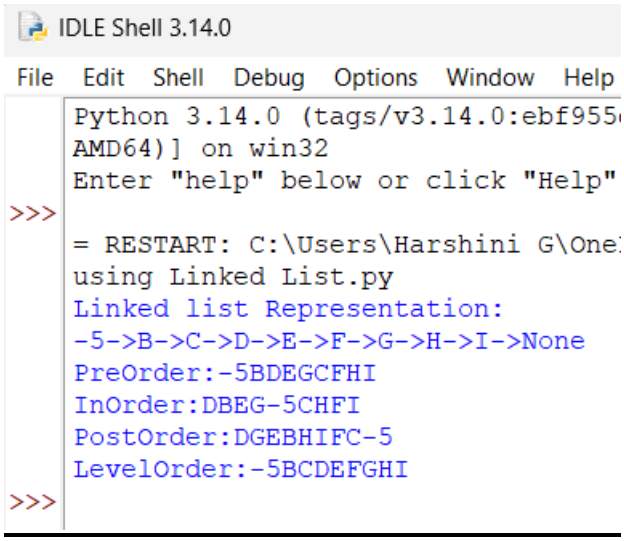
```
        while queue:
            node=queue.pop(0)
            print(node.data,end="->")
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)
        print("None")
tree=BinaryTree()
tree.create_tree()
print("Linked list Representation:")
tree.display()
print("PreOrder:",end="")
tree.preorder(tree.root)
print("\nInOrder:",end="")
tree.inorder(tree.root)
print("\nPostOrder:",end="")
tree.postorder(tree.root)
print("\nLevelOrder:",end="")
tree.levelorder()
```

OUTPUT(+VE):



OUTPUT(-VE):

If self.root=Node(-5) then, the output is:



Error Table:

Edge Case	Input	Error	Explanation
Invalid node argument	tree.preorder("A")	AttributeError	A string does not have left, right, or data attributes.

Explanation:

1) The program defines a Node class to store data and references to the left and right child nodes.

2) The BinaryTree class initializes the tree with an empty root (None).

3) The create_tree() function creates a binary tree with nodes A to I and connects them using left and right references.

4) The preorder() function traverses the tree in the order Root → Left → Right.

5) The inorder() function traverses the tree in the order Left → Root → Right, while postorder() follows Left → Right → Root.

6) The levelorder() function visits nodes level by level using a queue, and display() prints the nodes connected by arrows.

7) Finally, the program creates the tree and displays its linked representation and all four traversal outputs.

WEEK-9

9a) **AIM:** Write a python program to implement a binary search tree with its operations and traversals.

INPUT:

```
class Node:
    def __init__(self,data):
        self.data=data
        self.left=None
        self.right=None
# Insert a node
def insert(root,x):
    if root is None:
        return Node(x)
    if x<root.data:
        root.left=insert(root.left,x)
    elif x>root.data:
        root.right=insert(root.right,x)
    else:
        print("Duplicate value is not allowed")
    return root
# Preorder Traversal
def Preorder(root):
    if root is None:
        return
    print(root.data,end=" ")
    Preorder(root.left)
    Preorder(root.right)
# Inorder Traversal
def Inorder(root):
    if root is None:
        return
    Inorder(root.left)
    print(root.data,end=" ")
```

```
    Inorder(root.right)
# Postorder Traversal
def Postorder(root):
    if root is None:
        return
    Postorder(root.left)
    Postorder(root.right)
    print(root.data,end=" ")
# Find Minimum
def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root=root.left
    return root
# Find Maximum
def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root=root.right
    return root
# Search a node
def search(root,data):
    if root is None:
        return None
    if data<root.data:
        return search(root.left,data)
    elif data>root.data:
```

```
        return search(root.right,data)
    else:
        return root
# Count total nodes
def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left)+count_nodes(root.right)+1
# Count leaf nodes
def count_leaf(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaf(root.left)+count_leaf(root.right)
# Count internal nodes
def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left)+count_internal(root.right)+1
# Height of tree
def height(root):
    if root is None:
        return -1
    return max(height(root.left),height(root.right))+1
# Delete a node
def delete(root,x):
```

```
    if root is None:
        return None
    if x<root.data:
        root.left=delete(root.left,x)
    elif x>root.data:
        root.right=delete(root.right,x)
    else:
        # No children
        if root.left is None and root.right is None:
            return None
        # Only right child
        elif root.left is None:
            return root.right
        # Only left child
        elif root.right is None:
            return root.left
        # Two children
        successor=find_min(root.right)
        root.data=successor.data
        root.right=delete(root.right,successor.data)
    return root
# Level-order Traversal
def level_order(root):
    if root is None:
        return
    queue=[root]
    while queue:
        node=queue.pop(0)
        print(node.data,end=" ")
```

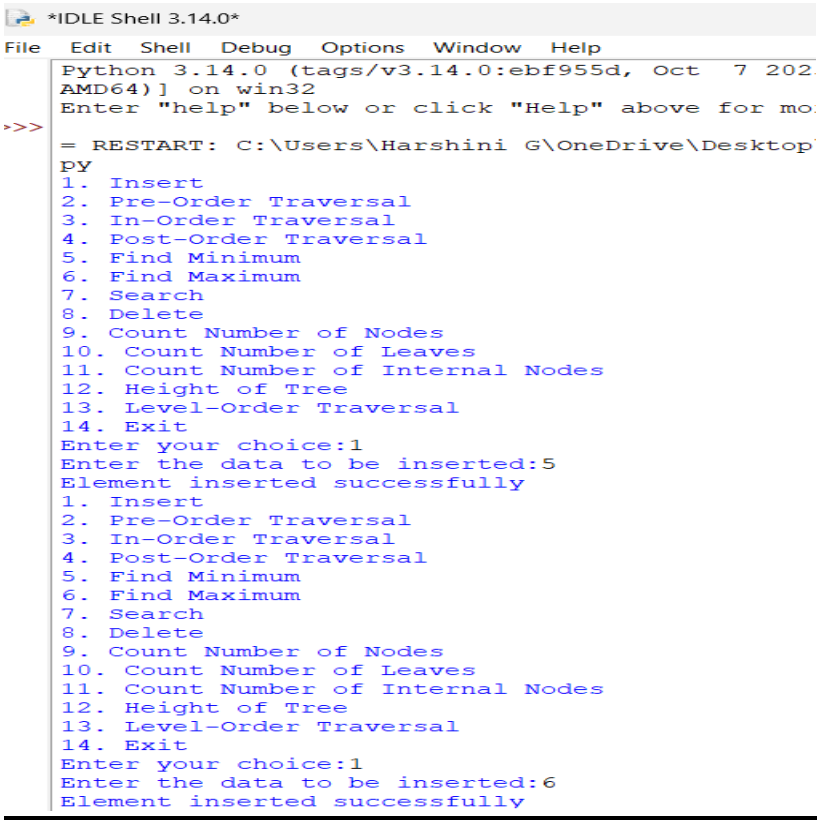
```
        if node.left is not None:
            queue.append(node.left)
        if node.right is not None:
            queue.append(node.right)
# Display tree level by level
def display(root,level=0):
    if root is None:
        return
    queue=[(root,0)]
    while queue:
        node,current_level=queue.pop(0)
        if current_level==level:
            print(node.data,end=" ")
        if node.left is not None:
            queue.append((node.left,current_level+1))
        if node.right is not None:
            queue.append((node.right,current_level+1))
# Menu-driven Program
root = None
while True:
    print("1. Insert")
    print("2. Pre-Order Traversal")
    print("3. In-Order Traversal")
    print("4. Post-Order Traversal")
    print("5. Find Minimum")
    print("6. Find Maximum")
    print("7. Search")
    print("8. Delete")
    print("9. Count Number of Nodes")
```

```
print("10. Count Number of Leaves")
print("11. Count Number of Internal Nodes")
print("12. Height of Tree")
print("13. Level-Order Traversal")
print("14. Exit")
choice=int(input("Enter your choice:"))
if choice==1:
    data=int(input("Enter the data to be inserted:"))
    root=insert(root,data)
    print("Element inserted successfully")
elif choice==2:
    print("Preorder Traversal:",end="")
    if root is None:
        print("Tree is Empty")
    else:
        Preorder(root)
        print()
elif choice==3:
    print("Inorder Traversal:",end="")
    if root is None:
        print("Tree is Empty")
    else:
        Inorder(root)
        print()
elif choice==4:
    print("Postorder Traversal:",end="")
    if root is None:
        print("Tree is Empty")
    else:
```

```
            Postorder(root)
            print()
        elif choice==5:
            node=find_min(root)
            if node is None:
                print("Tree is Empty")
            else:
                print("Minimum element:",node.data)
        elif choice==6:
            node=find_max(root)
            if node is None:
                print("Tree is Empty")
            else:
                print("Maximum element:",node.data)
        elif choice==7:
            data=int(input("Enter the data to be searched:"))
            node=search(root,data)
            if node is None:
                print("Element",data,"is not found")
            else:
                print("Element",data,"is found")
        elif choice==8:
            data=int(input("Enter the data to be deleted:"))
            root=delete(root,data)
        elif choice==9:
            print("Number of nodes:",count_nodes)
        elif choice==10:
            print("Number of leaf nodes:",count_leaf)
        elif choice==11:
```

```
        print("Number of internal nodes:",count_internal)
    elif choice==12:
        print("Height of BST:",height(root))
    elif choice==13:
        print("Level order Traversal:",end="")
        if root is None:
            print("Tree is Empty")
        else:
            level_order(root)
            print()
    elif choice==14:
        print("Program terminated.")
        break
    else:
        print("Enter a valid choice.")
```

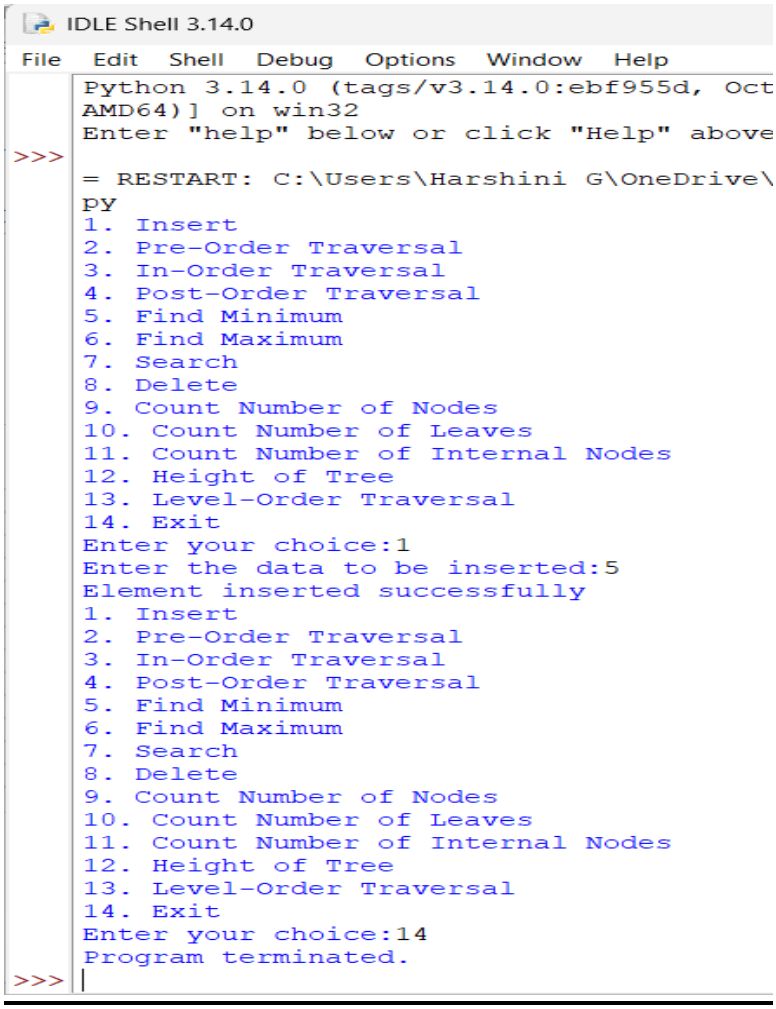
OUTPUT(+VE):



```
Element inserted successfully
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:1
Enter the data to be inserted:7
Element inserted successfully
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:2
Preorder Traversal:5 6 7
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
```

```
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:3
Inorder Traversal:5 6 7
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:4
Postorder Traversal:7 6 5
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:14
Program terminated.
```

OUTPUT(-VE):



Error Table:

Edge Case	Input	Error	Explanation
Non-integer node input	Choose option 1, enter abc	Value Error	BST node data must be an integer in this program.

Explanation:

1) The program defines a Node class to store data and references to the left and right child nodes.

2) The insert() function inserts elements into the Binary Search Tree (BST), placing smaller values on the left and larger values on the right.

3) The Preorder(), Inorder(), and Postorder() functions display the tree elements in different traversal orders.

4) The find_min(), find_max(), and search() functions find the minimum, maximum, and required element in the tree.

5) The count_nodes(), count_leaf(), and count_internal() functions count the total nodes, leaf nodes, and internal nodes, respectively.

6) The height() function calculates the tree height, while delete() removes a node and level_order() displays nodes level by level.

7) Finally, the menu-driven program allows the user to insert, search, delete, traverse, count nodes, find the height, and exit the program.

WEEK-10

10a) **AIM:** Write a python program to implement an AVL tree will all of its operation.

INPUT:

```
class Node:
    def __init__(self,data):
        self.data=data
        self.left=None
        self.right=None
        self.height=0
def height(root):
    if root is None:
        return -1
    return root.height
def balance(root):
    if root is None:
        return 0
    return height(root.left)-height(root.right)
def right_rotate(y):
    x=y.left
    a=x.right
    x.right=y
    y.left=a
    y.height=max(height(y.left),height(y.right))+1
    x.height=max(height(x.left),height(x.right))+1
    return x
```

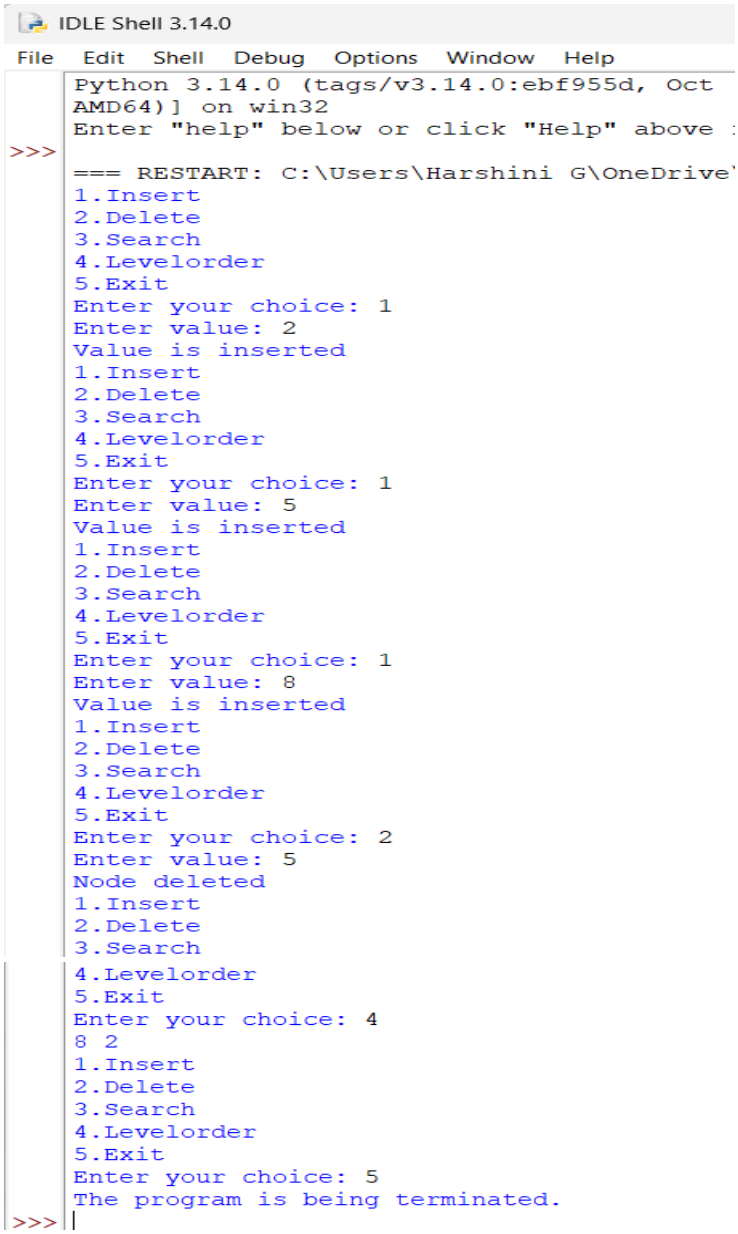
```
def left_rotate(x):
    y=x.right
    a=y.left
    y.left=x
    x.right=a
    x.height=max(height(x.left),height(x.right))+1
    y.height=max(height(y.left),height(y.right))+1
    return y
def insert(root,x):
    if root is None:
        return Node(x)
    if x<root.data:
        root.left=insert(root.left,x)
    elif x>root.data:
        root.right=insert(root.right,x)
    else:
        print("Duplicate values not allowed")
        return root
    root.height=max(height(root.left),height(root.right))+1
    b=balance(root)
    if b>1 and x<root.left.data:
        return right_rotate(root)
    if b<-1 and x>root.right.data:
        return left_rotate(root)
    if b>1 and x>root.left.data:
        root.left=left_rotate(root.left)
        return right_rotate(root)
    if b<-1 and x<root.right.data:
        root.right=right_rotate(root.right)
```

```
        return left_rotate(root)
    return root
def min_value(root):
    while root.left:
        root=root.left
    return root
def delete(root,key):
    if root is None:
        return root
    if key<root.data:
        root.left=delete(root.left,key)
    elif key>root.data:
        root.right=delete(root.right,key)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        temp=min_value(root.right)
        root.data=temp.data
        root.right=delete(root.right,temp.data)
    root.height=max(height(root.left),height(root.right))+1
    b=balance(root)
    if b>1 and balance(root.left)>=0:
        return right_rotate(root)
    if b>1 and balance(root.left)<0:
        root.left=left_rotate(root.left)
        return right_rotate(root)
    if b<-1 and balance(root.right)<=0:
```

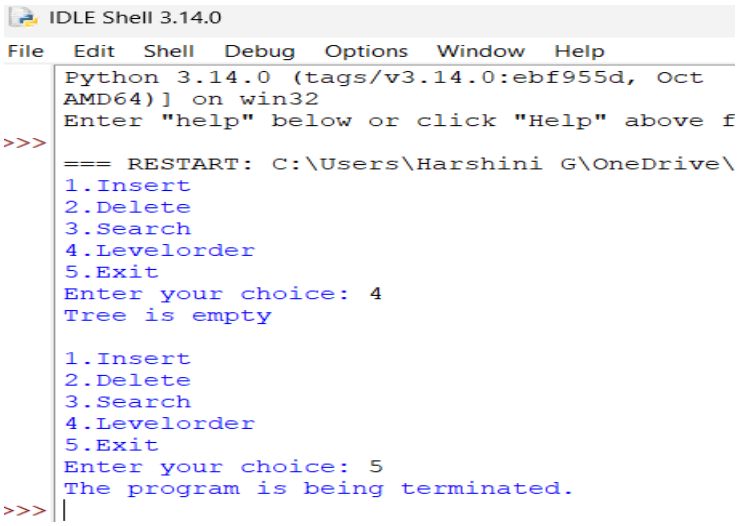
```
        return left_rotate(root)
    if b<-1 and balance(root.right)>0:
        root.right=right_rotate(root.right)
        return left_rotate(root)
    return root
def search(root,key):
    if root is None or root.data==key:
        return root
    if key<root.data:
        return search(root.left,key)
    return search(root.right,key)
def levelorder(root):
    if root is None:
        print("Tree is empty")
        return
    q=[root]
    while q:
        n=q.pop(0)
        print(n.data,end=" ")
        if n.left:
            q.append(n.left)
        if n.right:
            q.append(n.right)
root = None
while True:
    print("1.Insert")
    print("2.Delete")
    print("3.Search")
    print("4.Levelorder")
```

```
print("5.Exit")
ch = int(input("Enter your choice: "))
if ch==1:
    x=int(input("Enter value: "))
    root=insert(root,x)
    print("Value is inserted")
elif ch==2:
    x=int(input("Enter value: "))
    if search(root,x):
        root=delete(root,x)
        print("Node deleted")
    else:
        print("Value not found")
elif ch==3:
    x=int(input("Enter value: "))
    if search(root, x):
        print("Found")
    else:
        print("Not Found")
elif ch==4:
    levelorder(root)
    print()
elif ch==5:
    print("The program is being terminated.")
    break
else:
    print("Invalid choice")
```

OUTPUT(+VE):


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct
AMD64)] on win32
Enter "help" below or click "Help" above
>>>
=== RESTART: C:\Users\Harshini G\OneDrive
1.Insert
2.Delete
3.Search
4.Levelorder
5.Exit
Enter your choice: 1
Enter value: 2
Value is inserted
1.Insert
2.Delete
3.Search
4.Levelorder
5.Exit
Enter your choice: 1
Enter value: 5
Value is inserted
1.Insert
2.Delete
3.Search
4.Levelorder
5.Exit
Enter your choice: 1
Enter value: 8
Value is inserted
1.Insert
2.Delete
3.Search
4.Levelorder
5.Exit
Enter your choice: 2
Enter value: 5
Node deleted
1.Insert
2.Delete
3.Search
4.Levelorder
5.Exit
Enter your choice: 4
8 2
1.Insert
2.Delete
3.Search
4.Levelorder
5.Exit
Enter your choice: 5
The program is being terminated.
>>>

OUTPUT(-VE):


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct
AMD64)] on win32
Enter "help" below or click "Help" above f
>>>
=== RESTART: C:\Users\Harshini G\OneDrive\
1.Insert
2.Delete
3.Search
4.Levelorder
5.Exit
Enter your choice: 4
Tree is empty
1.Insert
2.Delete
3.Search
4.Levelorder
5.Exit
Enter your choice: 5
The program is being terminated.
>>>

Error Table:

Edge Case	Error	Explanation
Incorrect attribute used during searching.	Attribute Error	Changed root. key to root. data.

Explanation:

1. Node Creation: The Node class creates a node with data, left child, right child and height initialized to 0.

2. Height and Balance Factor: The height() function calculates the height of the tree, and balance() finds the difference between the left and right subtree heights.

3. Rotations: The left_rotate() and right_rotate() functions balance the AVL tree. Four cases are handled: LL, RR, LR and RL.

4. Insertion: The insert() function adds new nodes according to BST rules and performs rotations whenever the balance factor becomes greater than 1 or less than -1. Duplicate values are not allowed.

5. Deletion: The delete() function removes a node from the tree and updates its height and balance factor. It performs rotations if the tree becomes unbalanced.

6. Searching and Display: The search() function finds a particular element, while display() prints the tree in a structured format. The levelorder() function displays nodes level by level.

7. Menu-Driven Operations: The main program provides options for insertion, deletion, searching, level-order traversal, displaying the tree and exiting the program.

WEEK-11

11a) **AIM:** Write a python program to implement hashing (separate chaining, linear probing and quadratic probing).

INPUT:

```
def insert(heap,value):
    heap.append(value)
    i=len(heap)-1
    while i>0:
        parent=(i-1)//2
        if heap[i]>heap[parent]:
            heap[i],heap[parent]=heap[parent],heap[i]
            i=parent
        else:
```

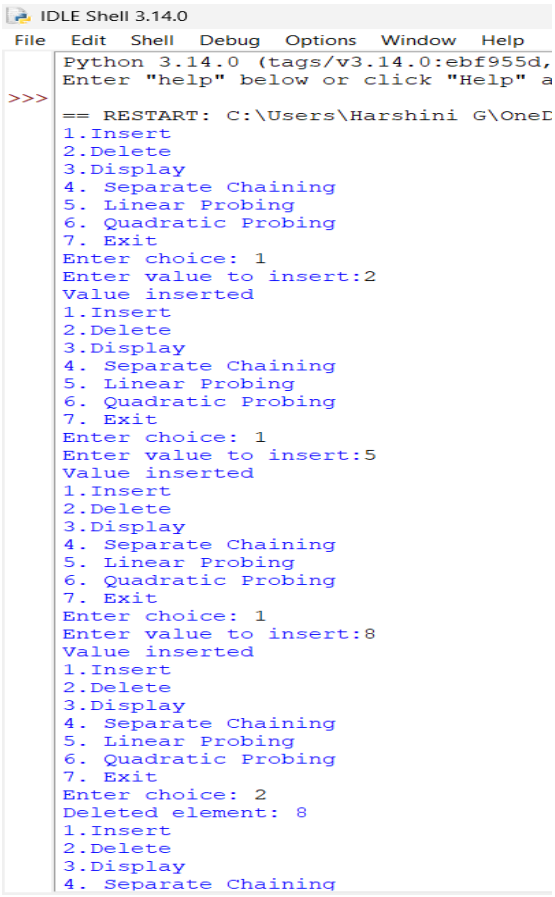
```
            break
def delete(heap):
    if len(heap)==0:
        print("Heap is empty")
        return
    print("Deleted element:",heap[0])
    heap[0]=heap[-1]
    heap.pop()
    n=len(heap)
    i=0
    #Move the root downward
    while True:
        largest=i
        left=2*i+1
        right=2*i+2
        if left<n and heap[left]>heap[largest]:
            largest=left
        if right<n and heap[right]>heap[largest]:
            largest=right
        if largest!=i:
            heap[i],heap[largest]=heap[largest],heap[i]
            i=largest
        else:
            break
def display(heap):
    if len(heap)==0:
        print("Heap is empty")
    else:
        print("Heap elements:",heap)
```

```
size=10
# Separate Chaining
def chaining():
    table=[[] for i in range(size)]
    n=int(input("Enter number of elements: "))
    for i in range(n):
        x=int(input("Enter value: "))
        index=x%size
        table[index].append(x)
    print("Hash Table:")
    for i in range(size):
        print(i,":",table[i])
# Linear Probing
def linear():
    table=[-1]*size
    n=int(input("Enter number of elements: "))
    if n>size:
        print("Hash table is full")
        return
    for i in range(n):
        x=int(input("Enter value: "))
        index=x%size
        count=0
        while table[index]!=-1:
            index=(index+1)%size
        count+=1
        if count==size:
            print("Hash table is full")
            return
```

```
            table[index]=x
        print("Hash Table:")
        for i in range(size):
            print(i,":",table[i])
    # Quadratic Probing
    def quadratic():
        table=[-1]*size
        n=int(input("Enter number of elements: "))
        for i in range(n):
            x=int(input("Enter value: "))
            index=x%size
            j=0
            while j<size and table[(index+j*j)%size]!=-1:
                j+=1
            if j==size:
                print("No empty slot found")
                return
            table[(index+j*j)%size]=x
        print("Hash Table:")
        for i in range(size):
            print(i,":",table[i])
    # Menu
    heap=[]
    while True:
        print("1.Insert")
        print("2.Delete")
        print("3.Display")
        print("4. Separate Chaining")
        print("5. Linear Probing")
```

```
print("6. Quadratic Probing")
print("7. Exit")
ch=int(input("Enter choice: "))
if ch==1:
    value=int(input("Enter value to insert:"))
    insert(heap,value)
    print("Value inserted")
elif ch==2:
    delete(heap)
elif ch==3:
    display(heap)
elif ch==4:
    chaining()
elif ch==5:
    linear()
elif ch==6:
    quadratic()
elif ch==7:
    print("Program is terminating")
    break
else:
    print("Invalid choice")
```

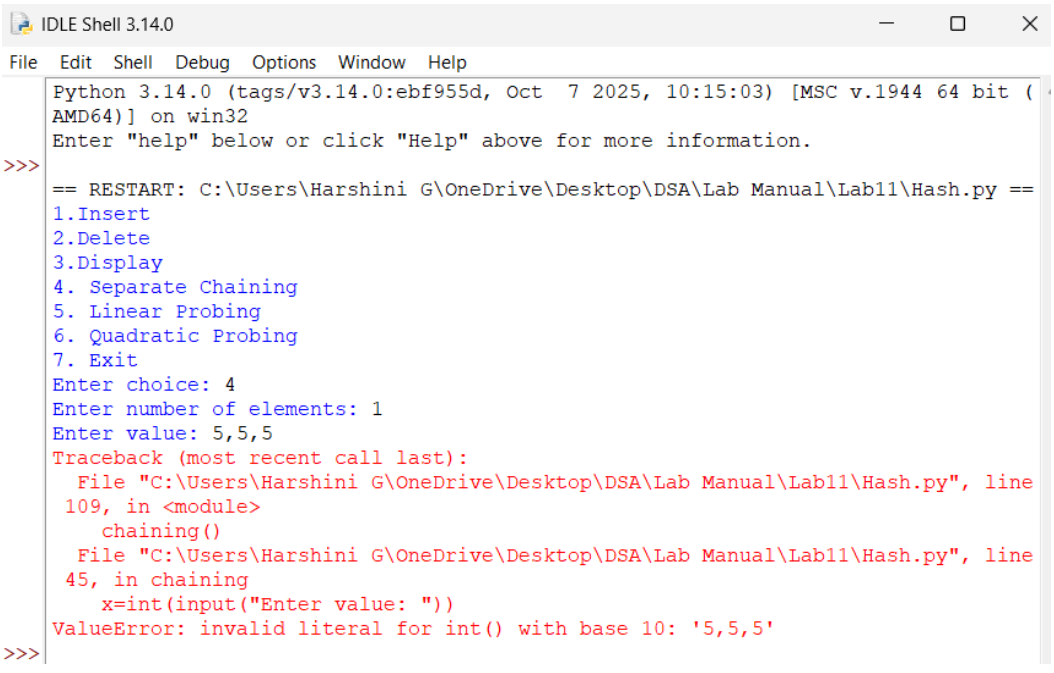
OUTPUT(+VE):




```
4. Separate Chaining
5. Linear Probing
6. Quadratic Probing
7. Exit
Enter choice: 3
Heap elements: [5, 2]
1.Insert
2.Delete
3.Display
4. Separate Chaining
5. Linear Probing
6. Quadratic Probing
7. Exit
Enter choice: 4
Enter number of elements: 2
Enter value: 2
Enter value: 3
Hash Table:
0 : []
1 : []
2 : [2]
3 : [3]
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []
1.Insert
2.Delete
3.Display
4. Separate Chaining
5. Linear Probing
6. Quadratic Probing
7. Exit
Enter choice: 5
Enter number of elements: 2
Enter value: 2
Enter value: 3
Hash Table:
0 : -1
1 : -1
2 : 2
3 : 3
4 : -1
5 : -1
```

```
1 : -1
2 : 2
3 : 3
4 : -1
5 : -1
6 : -1
7 : -1
8 : -1
9 : -1
1.Insert
2.Delete
3.Display
4. Separate Chaining
5. Linear Probing
6. Quadratic Probing
7. Exit
Enter choice: 6
Enter number of elements: 2
Enter value: 2
Enter value: 3
Hash Table:
0 : -1
1 : -1
2 : 2
3 : 3
4 : -1
5 : -1
6 : -1
7 : -1
8 : -1
9 : -1
1.Insert
2.Delete
3.Display
4. Separate Chaining
5. Linear Probing
6. Quadratic Probing
7. Exit
Enter choice: 7
Program is terminating
```

OUTPUT(-VE):



Error Table:

Edge Case	Error	Explanation
Entering a string instead of an integer	Runtime error (Value Error)	Enter valid integer input.

Explanation:

1. The program implements a Max Heap using a list and supports insertion, deletion, and display operations.

2. The insert() function adds an element and rearranges it upward to maintain the Max Heap property.

3. The delete() function removes the largest element (root) and rearranges the remaining elements downward.

4. The display() function prints the heap elements or displays a message if the heap is empty.

5. The chaining() function handles hash collisions by storing multiple elements in the same index using separate chaining.

6. The linear() and quadratic() functions resolve collisions using Linear Probing and Quadratic Probing, respectively.

7. The menu-driven program allows users to select operations repeatedly until they choose Exit.